Knowledge-Based Systems 328 (2025) 114192

Contents lists available at ScienceDirect

Knowledge-Based Systems

journal homepage: www.elsevier.com/locate/knosys

Dynamic scheduling for dual-resource constrained flexible job-shop via semantic-aware graph modelling and deep reinforcement learning

Yiwen Hu, Jie Chen, Zequn Zhang *, Dunbing Tang, Qixiang Cai

College of Mechanical and Electrical Engineering, Nanjing University of Aeronautics and Astronautics, Nanjing 210016, China

ARTICLE INFO

Keywords:
Knowledge graph
Graph attention network
Deep reinforcement learning
Dual-resource constraints
Dynamic scheduling

ABSTRACT

The dual-resource constrained flexible job-shop scheduling problem (DRCFJSP) poses significant challenges in dynamic manufacturing environments. Effective scheduling requires the integration of heterogeneous data from machines, workers, and operations and the ability to adapt to real-time changes. To address these challenges, this study proposes a hybrid scheduling method that combines knowledge graph (KG) techniques with deep reinforcement learning (DRL) to enable adaptive decision-making. A scheduling optimisation model is first formulated to minimise makespan and total production cost. To support real-time environmental perception, a manufacturing process dynamic knowledge graph (MPDKG) is constructed using a multi-level ontology framework. The MPDKG represents static attributes and dynamic states of scheduling entities, allowing continuous data updates during production. Semantic features are extracted using a semantic-aware heterogeneous graph attention network (SHGAT), which incorporates relation-aware and node-aware semantic aggregation to fuse multi-source data and capture global contextual information on operations, machines, and workers. The extracted features characterise the current state of the shop floor and serve as input to a multi-actor proximal policy optimisation (MA-PPO) algorithm. MA-PPO assigns specialised agents to different scheduling entities and learns coordinated decision policies through interaction with the environment. Experimental results from an aerospace structural component workshop show that the proposed method significantly outperforms baseline approaches in responsiveness and scheduling efficiency. These findings demonstrate the method's effectiveness and applicability in complex, dynamic production scenarios.

1. Introduction

With advancements in manufacturing equipment and evolving market demands, manufacturing systems are shifting from large-scale, assembly-line production to multi-species, variable-batch flexible production. This shift towards highly flexible production environments presents significant challenges for shop floor scheduling. As a critical component of the manufacturing process, scheduling directly impacts the efficient allocation of resources, production efficiency and cost reduction. Consequently, optimising scheduling strategies and effectively organising manufacturing resources have become essential for enterprises to remain competitive in the current wave of industrial transformation.

Flexible operation workshops, such as machining workshops, are increasingly tasked with customised, high-complexity production. In these environments, machines and workers must work closely, forming a dual-resource-constrained and highly flexible production setting. This complexity imposes greater demands on modelling and solving the workshop scheduling problems. Among the various resources, human workers are both the most important and flexible, but also the most vulnerable [1]. In response, the European Union has introduced the concept of Industry 5.0, building upon Industry 4.0, to emphasise human-centred manufacturing [2]. Within this context, the scheduling problem in dual-resource-constrained (DRC) production systems has emerged as a prominent research topic. Compared to the traditional flexible job-shop scheduling problem (FJSP), the dual-resource constrained flexible job-shop scheduling problem (DRCFJSP) [3] incorporates the heterogeneity and flexibility of workers. It consists of three subproblems: (1) machine assignment, (2) worker assignment, and (3) process sequencing. DRCFJSP more accurately reflects real-world production conditions in machine shops and offers valuable guidance for practical scheduling applications. However, prior studies often assume a one-to-one correspondence between workers and machines, ignoring the fact that, in practice, a worker may operate multiple

* Corresponding author.
E-mail address: zhjj370@nuaa.edu.cn (Z. Zhang).

https://doi.org/10.1016/j.knosys.2025.114192
Received 11 April 2025; Received in revised form 15 July 2025; Accepted 26 July 2025
Available online 31 July 2025
0950-7051/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

machines [4]. Although some researchers have considered factors such as worker cooperation [5] and skill levels [6] into the DRC scheduling framework, the impact of worker fatigue, a critical factor affecting productivity, has been largely ignored. To address these gaps, this study investigates the DRCFJSP with explicit consideration of both worker fatigue and skill level, and explores their combined effects on production efficiency.

The involvement of human workers introduces additional complexity and dynamism to shop floor operations. Traditional scheduling methods often fail to capture real-time changes in production information comprehensively and are unable to rapidly adjust schedules in response to these changes, thereby compromising the timeliness and effectiveness of the scheduling program. Consequently, the ability to unify the characterisation and fusion of the large amount of information generated by the elements of machines, workers, and processes in a dynamic workshop, while simultaneously capturing real-time changes, becomes essential for enabling rapid and adaptive adjustments to the production schedule.

In recent years, the manufacturing paradigm has been evolving towards cognitive intelligent manufacturing [7]. The strong potential of knowledge graphs (KG) in semantic representation and large-scale data integration has made them a key enabler of cognitive intelligence [8], attracting increasing attention from researchers. KG offer new opportunities for achieving comprehensive and accurate real-time insight into changes on the shop floor. The unified characterisation and fusion of diverse information, such as data, plans, and reports, generated throughout the manufacturing process form the foundation for real-time, data-driven dynamic scheduling. However, most existing research on KG in manufacturing tends to focus on isolated aspects, such as specific resources, processes or procedures. This narrow scope limits the ability to model the interrelationships among all elements and phases of shop floor operations. Moreover, the effective fusion of dynamic and static information remains a major challenge. Although some studies [9,10] have explored the use of KG in production scheduling, DRCFJSP presents additional complexity. In DRCFJSP, the various manufacturing elements are highly interdependent, placing greater demands on information modelling and integration. Therefore, developing an information fusion method tailored to DRCFJSP is necessary. Such a method should integrate heterogeneous data from all key manufacturing elements into a unified, reusable, and shareable knowledge model, thereby providing data support for advanced scheduling methods.

Based on the above analysis, this study addresses the practical challenges of dynamic DRCFJSP by considering key human factors, such as worker fatigue and skill levels. A dynamic scheduling method is proposed that integrates KG with deep reinforcement learning (DRL) to tackle issues such as poor real-time state tracking, low utilisation of shop floor information, weak integration of state features, and the complexity of solving dynamic scheduling problems. The main contributions of this work are as follows:

(1) A multi-objective integer programming model for DRCFJSP is developed, incorporating worker fatigue and proficiency to improve the applicability and accuracy of the scheduling model in real-world settings.
(2) To address the complexity and dynamic nature of DRCFJSP, a scheduling framework that integrates KG and DRL techniques is proposed, enabling flexible adaptation to production disturbances.
(3) A manufacturing process dynamic knowledge graph (MPDKG) is constructed to unify the representation of both static and dynamic shop floor elements, supporting continuous updates based on real-time production states.
(4) An end-to-end DRL algorithm is proposed, incorporating a semantic-aware heterogeneous graph attention network (SHGAT). SHGAT captures correlations among resources for effective KG graph embedding, which serves as the input to the DRL model for data-driven, dynamic, and optimised scheduling.

The remainder of this paper is structured as follows: Section 2 reviews the relevant literature. Section 3 formally defines the DRCFJSP, incorporating considerations of worker fatigue and recovery. Section 4 details the proposed framework for solving the DRCFJSP. Section 5 presents a case study based on a manufacturing workshop for aerospace structural components to verify the feasibility of the proposed method. Finally, Section 6 concludes the paper and outlines directions for future research.

2. Literature review

This section briefly reviews the conventional methods for solving DRC scheduling problems and the recent DRL approaches for addressing dynamic scheduling.

2.1. Knowledge modelling approach of manufacturing information

In manufacturing workshops, production elements generate complex, heterogeneous data and vast amounts of knowledge, making the integration of shop floor information increasingly challenging. Knowledge modelling offers a promising solution by enabling the unified representation of heterogeneous information and the abstract, normalised description of production elements such as resources, processes, and products. As research into knowledge modelling within the manufacturing domain has advanced, various approaches have been proposed to address issues of information consistency and accuracy. Object-oriented modelling [11] offers advantages in abstraction, stability and reusability; however, it lacks the capability to describe structured knowledge in detail. Petri net modelling [12], although useful for certain systems, suffers from poor readability in large-scale applications and struggles to characterise dynamic knowledge effectively. Ontology-based modelling [13] excels at expressing complex knowledge relationships and supporting interactions between heterogeneous systems, though it is not entirely equivalent to knowledge-based representation [14]. Sanfilippo et al. [15] utilised ontology modelling to represent manufacturing resources by aligning conceptual elements, such as activities, goals, and real-time events, thereby advancing the development of knowledge-based models for manufacturing resource information. Bao et al. [16] proposed a knowledge-based representation model grounded in ontology-driven digital twin modelling and evolution. Their approach integrates both resources and processes into a digital twin workshop and demonstrates the construction and evolution of the model over time.

Although the aforementioned studies have made significant progress in manufacturing information integration, limitations remain at the level of semantic representation. In recent years, KG has shown great potential in capturing semantic relationships among complex entities. Several studies have explored their application in modelling, fusion, and sharing manufacturing knowledge [17]. Duan et al. [18] developed an ontology model for the metal cutting process using the Web Ontology Language, and subsequently constructed a corresponding KG. Zhou et al. [9] proposed a resource KG-based allocation method to address emergency order insertion in discrete manufacturing workshops. Zhang et al. [19] proposed a subgraph semantic distance and ontology-based extraction method for modelling concepts, entities, and attributes related to metal materials, using KG to represent material knowledge effectively. However, most existing knowledge modelling methods in manufacturing primarily focus on static knowledge representation and management. Consequently, they struggle to capture the dynamic state transitions of resources in real-time. Although some efforts have begun to incorporate dynamic data, challenges related to data complexity, semantic relevance, and the impact of these factors on actual production processes have yet to be thoroughly addressed.

2.2. Dual-resource constrained scheduling problem

In DRC systems, approximately 60 %–90 % of failures are attributed to human workers [20]. Unlike machines, workers exhibit complex characteristics, such as varying skill levels, fatigue, and learning ability, which lead to fluctuating operational efficiency. To address these challenges, several studies have explored the role of human factors in scheduling optimisation. Song et al. [21] proposed a DRC distributed hybrid flowshop scheduling problem that accounts for worker fatigue, aiming to optimise makespan and energy consumption. They developed a Q-learning-based multi-objective evolutionary algorithm that demonstrated superior performance in both scheduling efficiency and energy use. Ren et al. [22] introduced a dual-resource optimisation configuration and collaborative scheduling method for DRCFJSP. Their method improves worker allocation, scheduling efficiency, and task distribution balance by quantifying worker flexibility, proficiency, and skill differences. In real production systems, worker productivity is affected by factors such as physical fatigue, rest allowance, and cooperative behaviour. Berti et al. [23] investigated the impact of an ageing workforce in JSP under fatigue conditions. Their findings indicate that allocating rest time based on workers' experience and fatigue level can significantly enhance productivity. For DRCFJSP scenarios involving worker cooperation, Luo et al. [24] proposed a Pareto-based two-stage evolutionary algorithm that optimises collaboration between workers and machines. This method reduces both makespan and workload, thereby improving overall scheduling efficiency.

In addition, owing to the harsh working environments, increasing levels of automation, and wages [25], the number of workers in actual workshops is often lower than the number of machines [26]. To address this imbalance, enterprises usually train workers in multi-skills. However, multi-skilled workers are heterogeneous in both their skill sets and proficiency levels [27]. An effective workforce can significantly reduce production costs while enhancing productivity and product quality [28]. Several studies have investigated scheduling and balancing problems involving multi-skilled workers. For the mixed-flow assembly line balancing problem, Zhang et al. [29] proposed a mixed-integer linear programming model and designed a novel multi-objective co-evolutionary algorithm aimed at minimising both energy consumption and costs. Liu et al. [30] examined the dynamic re-entry mixed-flow shop scheduling problem, considering the effects of worker fatigue and skill level. They proposed an integrated architecture based on deep reinforcement learning and a multi-agent system. Yu et al. [31] first introduced the energy-efficient distributed assembly hybrid flowshop scheduling problem, considering worker resources, formulating a mixed-integer linear programming model and solving it using a multi-objective memetic algorithm.

However, most existing current DRC scheduling studies consider workers as static resources, similar to machines, ignoring the heterogeneity of workers' fatigue, rest allowances, and other factors, and failing to analyse the specific effects of these factors on production in depth.

2.3. Dynamic scheduling method

Currently, shop floor dynamic scheduling methods can be broadly classified into three categories: robust scheduling [32], predictive-reactive scheduling (rescheduling) [33] and fully reactive (real-time) scheduling [34]. Robust scheduling improves stability by anticipating potential disturbances, but it lacks adaptability in highly dynamic environments. Predictive-reactive scheduling, often based on meta-heuristic algorithms, typically yields high-quality schedules but suffers from poor real-time performance. Traditional fully reactive scheduling relies on priority dispatching rules (PDRs) to make decisions based on real-time information. However, designing effective PDRs requires substantial domain expertise [35]. With the rise of end-to-end DRL methods, fully reactive scheduling is increasingly overcoming its dependence on manually crafted PDRss.

DRL methods for dynamic shop floor scheduling can be broadly categorised into DRL-based heuristics and end-to-end DRL methods. DRL-based heuristics combine DRL with traditional heuristic scheduling rules, aiming to balance computational speed with scheduling effectiveness. Luo [36] designed six composite scheduling rules as agent actions for dynamic FJSPs involving new workpiece insertion. A deep Q-network (DQN) was used to train the agent to select the most appropriate rule at each rescheduling point, with the goal of minimising total latency. Chang et al. [37] proposed a double deep Q-network (DDQN) architecture to solve dynamic FJSPs characterised by randomly arriving jobs. Although these approaches enhance adaptability, they still rely heavily on the design of dispatching rules, limiting their generalisation, similar to traditional fully reactive scheduling. In contrast, end-to-end DRL methods usually leverage a graph neural network (GNN) to extract structured features from shop floor data and construct scheduling policies directly through DRL, offering better generalisation capabilities. For instance, to address JSPs with random processing time and machine failures, Liu et al. [38] proposed a hybrid graph transformer network for extracting embeddings of coupled dynamic events, along with an auxiliary learning phase using PPO to refine the scheduling policy. Lei et al. [39] developed a multi-agent hierarchical RL framework based on parsed graphs to minimise the makespan in large-scale dynamic FJSPs with random job arrivals. Zhang et al. [40] introduced an end-to-end GNN-based DRL approach for dynamic FJSPs under transportation resource constraints. In summary, DRL-based dynamic scheduling methods have demonstrated promising performance across various problem scales. However, their application to real-world dynamic scheduling, particularly for the DRCFJSP, remains limited. To date, no comprehensive solution has been proposed to address the unique challenges of DRCFJSP using DRL.

2.4. Research gap

The existing literature highlights several key gaps in the domain of dynamic scheduling for manufacturing systems, which this study aims to address:

(1) Knowledge modelling for manufacturing information: Traditional knowledge modelling methods predominantly focus on static data representations, limiting their effectiveness in dynamic production environments. To overcome this limitation, our approach integrates real-time, dynamic data into a KG, enabling more accurate, responsive, and adaptive decision-making within manufacturing systems.
(2) Dual-resource constrained scheduling: Many prior studies treat workers as static resources, overlooking essential human-related factors such as fatigue, skill levels, and rest allowances. Our method explicitly models these dynamic worker characteristics, offering a more realistic and effective solution to the dual-resource-constrained scheduling problems.
(3) Dynamic scheduling methodology: Existing dynamic scheduling techniques largely depend on PDRs or DRL-based heuristics, which often lack robustness in responding to real-time changes. Furthermore, no in-depth research has been conducted on DRCFJSP. To fill this void, we propose an end-to-end scheduling framework that combines DRL with GNNs, delivering a more flexible, scalable and adaptive solution suited to the complexities of real-time manufacturing environments.

In conclusion, this study addresses these critical research challenges by integrating dynamic shop floor data, modelling worker heterogeneity, and developing a fully adaptive scheduling framework that significantly enhances the responsiveness and real-time dynamic manufacturing environments.

Table 1
List of notations.

Notation	Description
J_i	The ith job.
O_{ij}	The jth operation of J_i.
M_k	The kth machine.
W_r	The rth worker.
p_{ijk}	The preset processing time of the O_{ij} on M_k.
p_{ijkr}	The actual processing time of the O_{ij} on M_k by W_r.
δ^k	The degree of influence of fatigue level on processing time on the M_k.
λ^{kr}	The fatigue accumulation rate parameter of W_r working on M_k.
μ^r	The fatigue recovery rate parameter of W.
me_k	The machine parameter of M_k.
we_r	The worker parameter of W_r
β_f, γ_f	The fatigue recovery constant.
$FL_r(t)$	The fatigue level of W_r at time t.
PL_{ijr}	The skill level parameter of the W_r for O_{ij}.
MC_k	The unit processing cost of M_k.
WC_{ijr}	The unit processing cost of W_r processing O_{ij}.
AT_i	The arrival time of J_i.
S_{ij}	The start processing time of O_{ij}.
C_{ij}	The completion time of O_{ij}.
x_{ijkr}	If O_{ij} is processed by W_r on M_k, it is 1; otherwise, it is 0.
$y_{iji'j'k}$	If O_{ij} is processed on M_k before $O_{i'j'}$, it is 1; otherwise, it is 0.
$z_{iji'j'r}$	If O_{ij} is processed by W_r before $O_{i'j'}$, it is 1; otherwise, it is 0.

3. Problem description and model

This section describes the problem and mathematical modeling of the DRCFJSP considering the level of worker fatigue and skill level.

3.1. Problem description

DRCFJSP is defined as follows: There are n jobs $J = \{J_1, J_2, ..., J_n\}$ to be processed on m machines $M = \{M_1, M_2, ..., M_m\}$ by l workers $W = \{W_1, W_2, ..., W_l\}$. The process set $O_i = \{O_{i1}, O_{i2}, ..., O_{in_i}\}$ of the job J_i consists of n_i processes that follow the sequence. O_{ij} is the jth process of the J_i, processed on a machine $M_k \in M_{ij}$ by a worker $W_r \in W_{ij}$, and M_{ij} and W_{ij} are the set of candidate machines and the set of candidate workers for O_{ij}, respectively. The W_{ij} fatigue level is $FL_r(t)$, and the skill level parameter for processing O_{ij} is PL_{ijr}. The preset processing time of O_{ij} on M_k is p_{ijk}, and the actual processing time is $p_{ijkr}$is determined by the $FL_r(t)$ and PL_{ijr} of the selected W_r, and jobs arrive randomly during the production process. The parameters are shown in Table 1.

The following assumptions are made:

(1) At time $t = 0$, all machines, workers, and operations are available, and all workers have a fatigue level of 0.
(2) At any given time, a machine and a worker can each process only one operation.
(3) There are no priority constraints among jobs.
(4) Machine failures and non-machining processes are not considered in this study.

3.2. Model of the influence of fatigue and skill level on processing efficiency

3.2.1. Worker fatigue level

In production workshops, machine operability is a primary factor contributing to worker fatigue accumulation [41]. The rate of fatigue recovery is influenced by the workshop environment and physiological characteristics of the workers [42]. Jaber et al. [43] emphasised that the fatigue model initially proposed by Konz [44] offers a more realistic representation of fatigue dynamics and has been widely adopted in scheduling research. Building on this foundation, this study incorporates both machine characteristics [45] and worker-specific factors [46] to refine the mathematical model of fatigue accumulation and recovery.

The fatigue level of a worker after performing a task for t_{i-1} to t_i continuous work/rest is calculated as follows:

$$FL_r(t_i) = \begin{cases} FL_r(t_{i-1}) + (1 - FL_r(t_{i-1}))\left(1 - e^{-\lambda^{kr}(t_i - t_{i-1})}\right), & working \\ FL_r(t_{i-1})e^{-\mu^r(t_i - t_{i-1})}, & resting \end{cases} \quad (1)$$

where λ^{kr} is the fatigue accumulation rate of W_r working on machine M_k, and μ^r is the fatigue recovery rate of W_r; the greater the value, the faster the accumulation/recovery speed. λ^{kr} and μ^r are calculated according to Eqs. (2) and (3), respectively:

$$\lambda^{kr} = me_k \times we_r \quad (2)$$

$$\mu^r = (\beta_f - we_r)\gamma_f \quad (3)$$

where me_k and we_r are the coefficient of M_k and W_r, respectively. me_k is positively correlated with M_k operation difficulty and we_r is negatively correlated with W_r physical condition. β_f and γ_f are constants.

3.2.2. Worker skill level

A worker may possess the ability to process multiple operations but with varying skill levels. Skill levels are categorised into three classes based on the complexity of the operation: primary I, intermediate II and advanced III. For a given operation, a worker maintains the same skill level across different machines; however, different workers may have different skill levels for different operations. Additionally, a single worker may have different skill levels for different operations. The skill levels and unit processing cost of W_r processing O_{ij} are defined as parameters: $WC_{ijr} = \{[a_{\mathrm{I}}, b_{\mathrm{I}}], [a_{\mathrm{II}}, b_{\mathrm{II}}], [a_{\mathrm{III}}, b_{\mathrm{III}}]\}$ and $PL_{ijr} = \{p_{\mathrm{I}}, p_{\mathrm{II}}, p_{\mathrm{III}}\}$, respectively. The larger the PL_{ijr}, the higher the skill level of W_r and corresponding WC_{ijr}.

3.2.3. Influence of fatigue level and skill level on processing efficiency

Based on the above fatigue model and skill level, Eq. (4) is used to correct the predefined processing time p_{ijk} and calculate the actual processing time p_{ijkr} of the O_{ij} under the influence of the fatigue level and skill level of workers:

$$p_{ijkr} = \frac{p_{ijk}\left(1 + \delta^k(\ln(1 + FL_r(t)))\right)}{PL_{ijr}} \quad (4)$$

where the parameter δ^k was used to define the influence of fatigue on the processing time. The automation level of M_k was associated with a negative correlation. When M_k was fully automated, the value was 0, indicating that fatigue did not affect the processing.

3.3. Mathematical model of the DRCFJSP

This paper constructs the following DRCFJSP model with the optimisation goal of minimising the maximum completion time C_{max} and minimising the total cost L_{cost}:

Objective:

$$min\ f_1 = C_{max} = max_{i,j} C_{ij}, \forall i, j \quad (5)$$

$$min\ f_2 = L_{cost} = \sum_{i=1}^{n}\sum_{j=1}^{n_i}\sum_{k=1}^{m}\sum_{r=1}^{l}\left(MC_k + WC_{ijr}\right) \times x_{ijkr} \times p_{ijkr} \quad (6)$$

Subject to:

$$C_{ij} \geq S_{ij} + p_{ijkr} - \left(1 - x_{ijkr}\right) \times N, \forall i, j, k, r \quad (7)$$

$$S_{ij} \geq C_{i(j-1)}, \forall i, j = 2, \cdots, n_i \quad (8)$$

$$C_{i'j'} + \left(1 - y_{iji'j'k}\right) \times N \geq C_{ij} + \sum\nolimits_{r=1}^{l} p_{i'j'kr} x_{i'j'kr}, \forall i, j, \forall M_k \in M_{ij} \cap M_{i'j'} \quad (9)$$

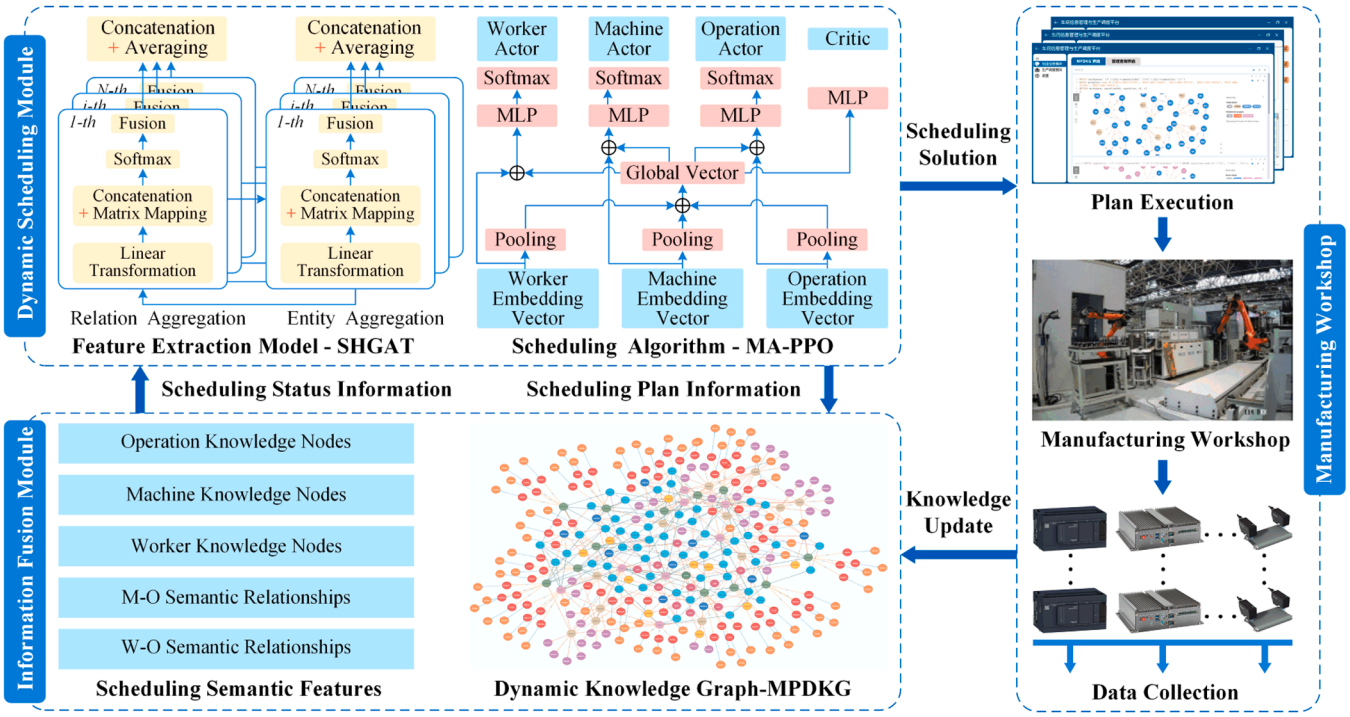


Fig. 1. Framework of proposed method.

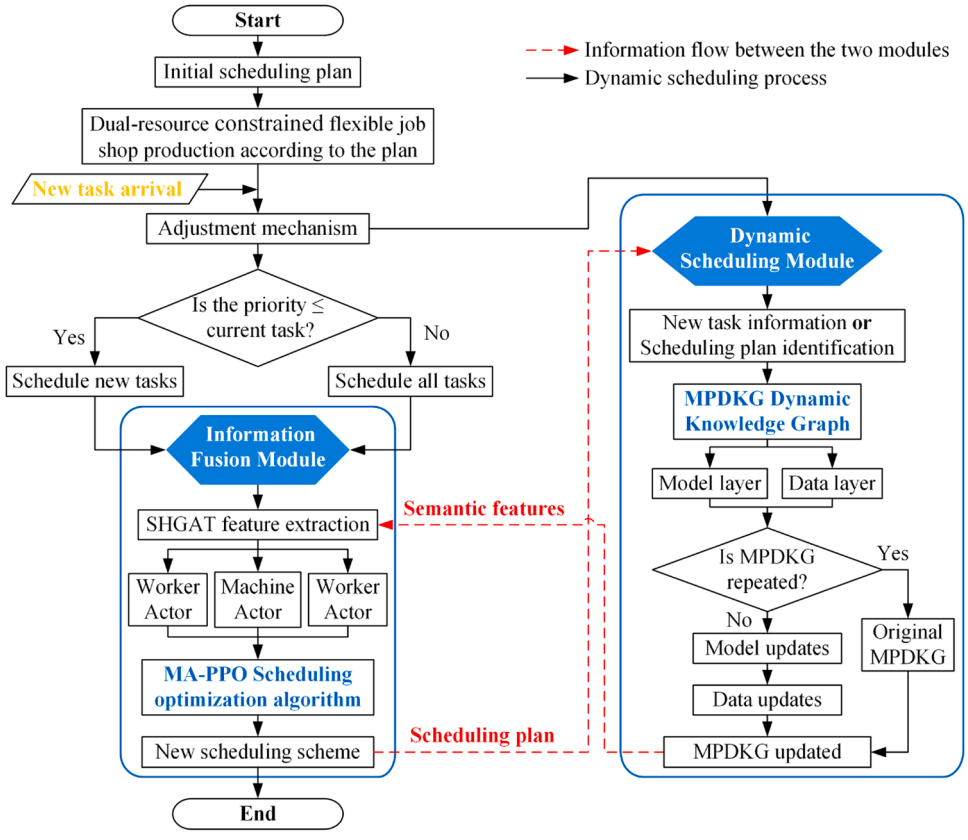


Fig. 2. Workflow of dynamic scheduling for DRCFJSP based on KG and DRL.

$$C_{i'j'} + \left(1 - z_{iji'j'k}\right) \times N \geq C_{ij} + \sum_{k=1}^{m} p_{i'j'kr} x_{i'j'kr}, \forall i,j, \forall W_r \in W_{ij} \cap W_{i'j'} \quad (10)$$

$$\sum_{k}^{m} \sum_{r}^{l} x_{ijkr} = 1, \forall i,j \quad (11)$$

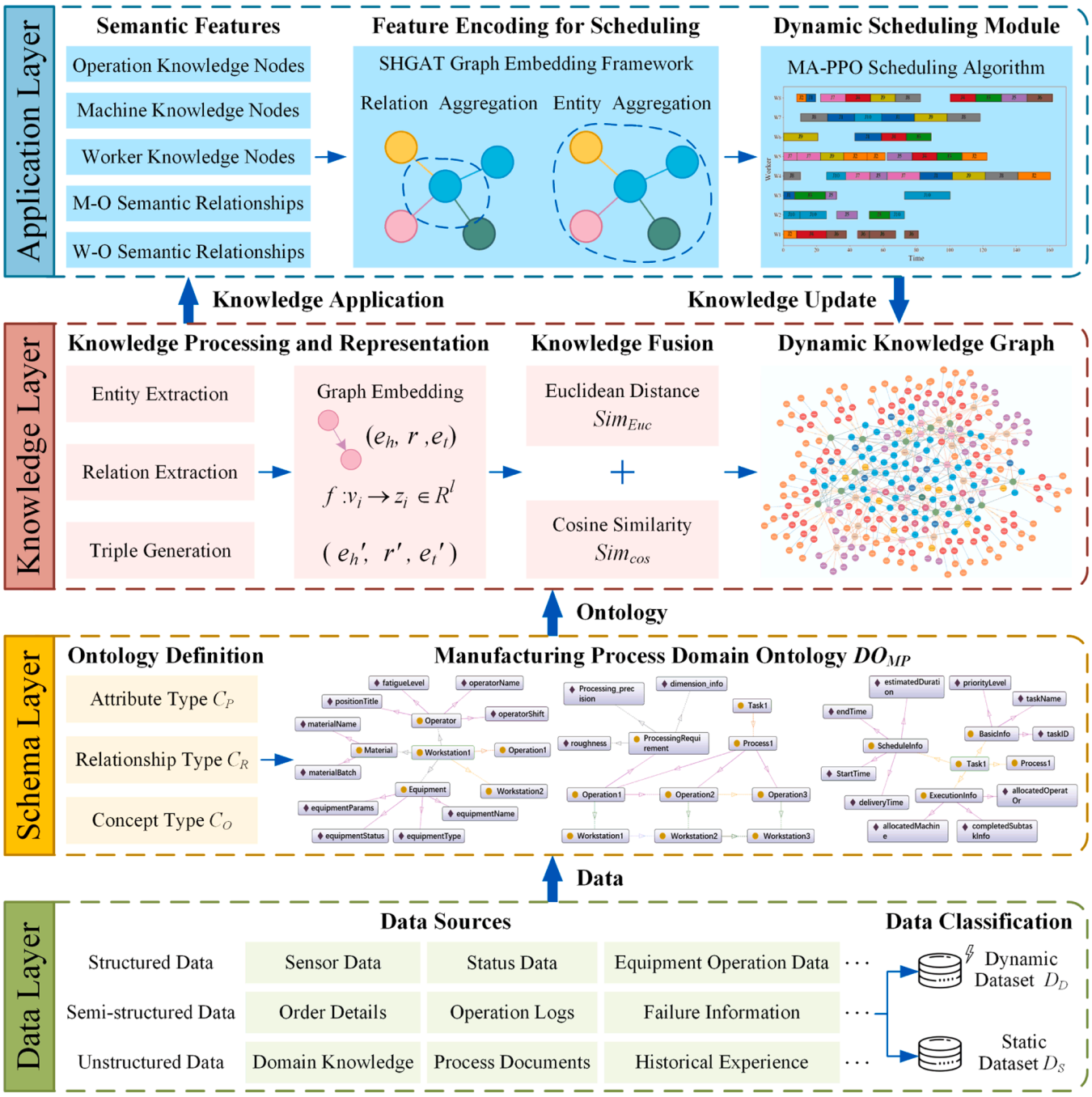


Fig. 3. Overall framework of the manufacturing process dynamic knowledge graph.

$$x_{ijkr} \in \{0,1\}, \forall i,j,k,r \tag{12}$$

$$y_{iji'j'k} \in \{0,1\}, \forall i,j,i',j', \forall M_k \in M_{ij} \cap M_{i'j'} \tag{13}$$

$$z_{iji'j'r} \in \{0,1\}, \forall i,j,i',j', \forall W_r \in W_{ij} \cap W_{i'j'} \tag{14}$$

Eqs. (5) and (6) define the two objective functions: minimising makespan and total production cost, respectively. Eq. (7) describes the relationship between the start and end times of O_{ij}, based on its actual processing time. N is a large positive number, and ensures equality only if its processing resources are M_k and W_r. Eq. (8) restricts the processing sequence within the same job, ensuring that the start time of each subsequent operation occurs after the completion of the previous one. Eq. (9) limits each machine to process only one operation at a time, whereas Eq. (10) imposes the same restriction on worker. Eq. (11) ensures that each operation is processed exactly once. Eqs. (12)–(14) define the feasible ranges and binary nature of the decision variables. The proposed model is applicable across industries. Although the case study in this paper focuses on an aerospace manufacturing workshop, the underlying principles are generalisable to any DRC manufacturing system. Industries with dynamic scheduling demands can also benefit from the flexibility and adaptability offered by the proposed approach.

4. Proposed method

This section introduces a DRCFJSP, which integrates a KG with DRL. Additionally, a SHGAT is proposed to perform graph embedding and establish an effective relationship between scheduling information and

Table 2
Definition and representation of semantic relations.

Type	Name	Definition & Example	Representation
Characteristic relationship R_C	*Has*	Sub-entity relationship of an entity node, indicating the entity has a static sub-entity	$e_h \longrightarrow^{Has} e_t$
	HasAttr	Attribute relationship of an entity node, indicating the entity has a static attribute	$e_h \longrightarrow^{HasAttr} e_t$
	Is	Attribute value relationship of an attribute node, indicating the attribute has a static value	$e_h \rightarrow^{Is} e_t$
Sequential relationship R_S	*NextTo*	Constraint relationship between process nodes in the manufacturing flow	$e_h \longrightarrow^{NextTo} e_t$
	FlowTo	Flow relationship between workstation nodes in a job	$e_h \longrightarrow^{FlowTo} e_t$
Temporal relationship R_T	*Has* : *T*	At time T, the entity node has a dynamic sub-entity	$e_h \longrightarrow^{Has:T} e_t$
	Is : *T*	At time T, the attribute node has a dynamic attribute value	$e_h \longrightarrow^{Is:T} e_t$

scheduling algorithms.

4.1. DRCFJSP dynamic scheduling framework integrating KG and DRL

Owing to the discrete nature of manufacturing processes, the diversity of resources, and the complexity of data flows, real-time information in the DRCFJSP environments is often poorly integrated and underutilised. To address this issue, the proposed framework aims to bridge heterogeneous manufacturing data with scheduling algorithms, enabling real-time, data-driven decision-making. As illustrated in Fig. 1, the framework consists of two main components: a KG-based information fusion module and a DRL-based dynamic scheduling module. The information fusion module builds an MPDKG, which integrates and represents heterogeneous and multi-source shop-floor data. The MPDKG is continuously updated as production progresses. When a new or unexpected task arises, the MPDKG immediately captures the relevant information and generates semantic features that describe both static attributes and dynamic states required for scheduling. These semantic features are then passed to the dynamic scheduling module. Within this module, SHGAT is used to extract high-quality embeddings of operations, machines, and workers from the MPDKG. These embeddings are used to construct actor networks and a critic network, which are trained using the MA-PPO algorithm. The trained model generates adaptive scheduling plans based on the current system state and dynamically responds to real-time events. As scheduling instructions are executed on the shop floor, the MPDKG is simultaneously updated to reflect the latest state of the system.

The specific dynamic scheduling process is illustrated in Fig. 2. When an initial task is released, the DRL-based dynamic scheduling module generates a preliminary scheduling plan, which is then used to update the MPDKG. At time t, the arrival of a new task triggers the disturbance response mechanism. The MPDKG within the KG-based information fusion module is updated to incorporate the new task information, whereas the DRL-based module assesses the task's priority. Next, the updated scheduling-related information encoded within the MPDKG is processed by SHGAT, which extracts semantic features representing the current production state. These features are then fed into the MA-PPO algorithm to generate an optimised scheduling plan that accommodates the new disturbance. Finally, the MPDKG is updated once more based on the new scheduling results, ensuring that it accurately reflects the current state of the shop floor. Compared to traditional scheduling approaches, the proposed method incorporates a dynamic MPDKG update mechanism, establishing a tight integration between the information fusion and dynamic scheduling modules. This close coupling improves the system's ability to perceive and respond to changes in the production environment in real time, thus enabling efficient dynamic scheduling in data-driven DRCFJSP scenarios.

4.2. Construction of KG-based information fusion module

4.2.1. MPDKG overall framework

MPDKG is defined as follows:

$$MPDKG = \langle E_{Entity}, C_R, F \rangle \tag{15}$$

where E_{Entity} is the set of entities, including conceptual entities, instance entities, and attribute-valued entities. It is composed of two types of nodes: static nodes E_S and dynamic nodes E_D. The static nodes are derived from static data sources, such as attribute-value pairs representing machine names. Dynamic nodes, conversely, are constructed from dynamic data sources, such as spindle speeds of machine tools, which change during operation. Let C_R is the set of all relation types. F is the set of all 'node-relationship-node' triples $F = \{(e_h, r, e_t) | e_h, e_t \in E_{Entity}, r \in C_R\}$, and e_h is the head entity, r is the relation, and e_t is the tail entity.

MPDKG is a typical domain-specific KG, constructed using a top-down methodology [47]. The overall framework is shown in Fig. 3. The data layer contains various types of data that reflect the real-time state of the manufacturing environment. In the data layer $D = \langle D_S, D_D \rangle$, D_S is the static data set, used to describe the basic information of manufacturing resources. This includes unstructured data such as process documentation and operation logs. D_D is the dynamic dataset, which captures real-time information from the manufacturing floor. This includes structured data such as equipment status and production schedule. Pattern layer consists of three ontologies, static resource, manufacturing process and processing task, which collectively specify concepts, relationships and constraints. These ontologies ensure consistency, standardisation and interoperability of the knowledge structure. The knowledge layer is where the MPDKG is instantiated by integrating both static and dynamic data. The dynamic update mechanism ensures that the MPDKG accurately reflects the current state of the manufacturing shop floor, enabling real-time responsiveness in the scheduling system.

4.2.2. Ontology-based multilevel knowledge modelling

The manufacturing process domain ontology is defined as:

$$DO_{MP} = \langle C_O, C_R, C_P \rangle \tag{16}$$

where $C_O = \langle O_R, O_P, O_T \rangle$ is a concept set, O_R is static resource ontology, O_R is manufacturing process ontology, O_T is processing task ontology, $C_R = \langle R_C, R_S, R_T \rangle$ is a relation set, as shown in Table 2, the characteristic relation R_C describes the basic attributes of nodes, the order relation R_S constrains the sequence of nodes, the temporal relation R_T reflects the dynamic attributes with time characteristics. reflects the dynamic attributes with temporal characteristics. C_P is the set of attributes describing the characteristics of each node.

Based on three ontology types defined by DO_{MP}, a hierarchical multilevel knowledge model is constructed, as illustrated in Fig. 4. The bottom layer, static resource ontology O_R describes the basic static information such as resources, materials and environment with the core of workstations; the middle layer manufacturing process ontology O_P carries the dynamic parameters related to machining process and process steps in the manufacturing process; and the top layer machining task ontology O_T characterises the basic information of the task, the process requirements, and the data with the temporal characteristics.

4.2.3. MPDKG construction and integration

Neo4j is a graph-based database designed for the efficient processing and querying of complex relational data. In this study, Neo4j is selected as the storage and query platform for the MPDKG, owing to its advantages in handling graph-structured data and supporting semantic relationships. Neo4j supports two core data types: *Nodes* and *Relationships*,

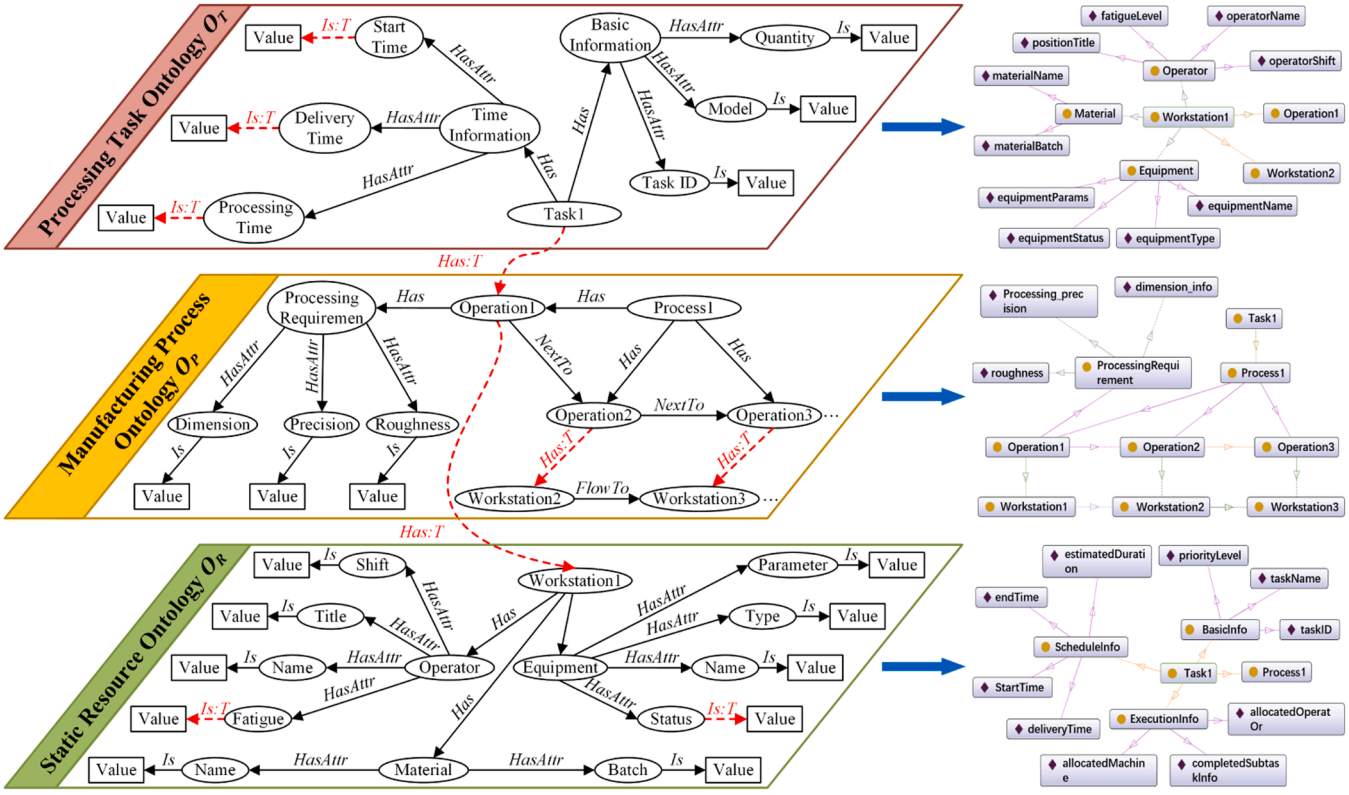


Fig. 4. Ontology-based multilevel knowledge modeling.

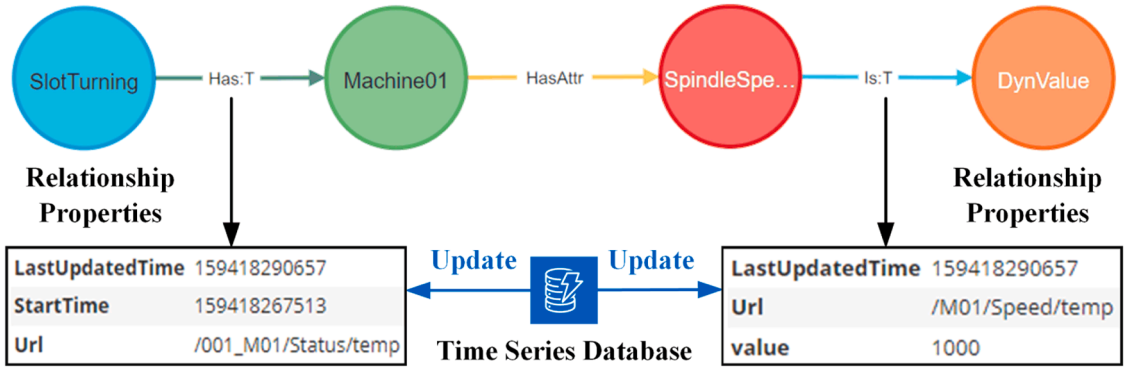


Fig. 5. Dynamic node attribute construction method in Neo4j.

whose attributes and their corresponding values are stored as key-value pairs in *Node Properties* and *Relationship Properties*, respectively, in the form {Attribute: Attribute Value}. Therefore, Neo4j is chosen as the MPDKG storage tool, and when constructing the attributes of static node E_S generated by static data D_S, the feature relations *HasAttr* and *Is* are omitted, and {Attribute: Attribute Value} is stored in *Nodes Properties*, resulting in a more concise and intuitive representation within the MPDKG. Construct nodes for all dynamic nodes E_D that are only used to declare "DynValues" that do not store data, and the actual dynamic attributes and values are stored as {Attribute: Attribute Value} in the *Relationship Properties* of the temporal relationship *Is* : *T*, including the timestamps LastUpdatedTime and DataSourceUrl as well as several dynamic information. As shown in Fig. 5, a 'DynValue' node is defined for the dynamic property node (SpindleSpeed) of the entry node (machine01), whereas the actual property value is stored in the *Relationship Properties* of *Is* : *T*. The corresponding dynamic data D_D is stored as key-value pairs in the *Relationship Properties* of *Has* : *T*. The ternary {SlotTurning, *Has* : *T*, Machine01} indicates that SlotTurning occupies Machine01, and the relevant time information is stored in the *Relationship Properties* of *Has* : *T*.

The construction process of the MPDKG involves several key steps: First, structured data is mapped to entities and relationships based on the ontologies defined in the pattern layer. This produces subject–predicate–object triples in the form of semantic statements. For example: {(subject: station 1), (predicate: *Has*), (object: CNC machine 1)}.

Next, the raw text is segmented into words using the cws.model, followed by part-of-speech tagging with pos.model. Named entity recognition (NER) is applied using ner.model, and syntactic dependency parsing is conducted using parser.model. Based on the dependency results, predicate triples are extracted. For example, from a sentence like 'Machine selection should consider average utilisation', the following triple may be extracted: {subject: machine selection), (predicate: consider), (object: average utilisation)}. Subsequently, a connection to

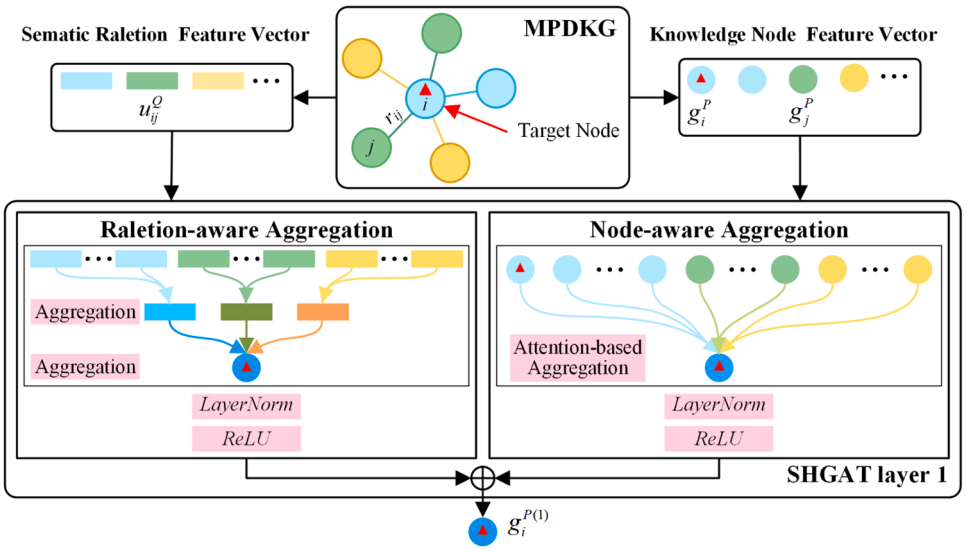


Fig. 6. Illustration of the SHGAT graph embedding process.

the Neo4j database is established via Python. The extracted triples are then organised according to the predefined multi-layer ontology model and inserted intoNeo4j using the cypher query language (CQL). During query execution, the system verifies the validity of dynamic attribute values by checking their timestamps. If the time elapsed since the last recorded timestamp exceeds a predefined threshold, the corresponding data is marked as outdated. In such cases, the system automatically retrieves the updated values from an external time-series database to ensure the freshness and accuracy of the MPDKG.

The constructed MPDKG contains a large number of shared nodes across multiple domains, leading to potential redundancy. To address this, knowledge fusion techniques are employed to consolidate overlapping entities into unified representations, thereby enhancing consistency and reducing duplication [48]. For knowledge representation, the TransE embedding model is adopted [49]. Through uniform distribution $U(-\epsilon,\epsilon)$ as $e_i \in E_{Entity}$ and $r_i \in C_R$ generated initial vector and normalised $\overrightarrow{e_i} = \overrightarrow{e_i}/\overrightarrow{e_i}_2 \in R^d$ and $\overrightarrow{r_i} = \overrightarrow{r_i}/\overrightarrow{r_i}_2 \in R^d$, the scoring function φ is as follows:

$$\varphi(e_h, r, e_t) = \overrightarrow{e_h} + \overrightarrow{r} - \overrightarrow{e_t}_2^2 \tag{17}$$

where for each positive triple(e_h,r,e_t), a set F' of negative samples (e_h', r', e_t') is generated by replacing either the relation r or the tail entity e_t, and γ is a margin. The entity and relation embeddings $\overrightarrow{e_i}$ and $\overrightarrow{r_i}$ are iteratively updated using stochastic gradient descent and normalised until convergence. Subsequently, semantic similarity between entities is computed to perform entity alignment. To improve the accuracy of the alignment process, a hybrid similarity computation method is proposed, which combines cosine similarity and Euclidean distance as follows:

$$Sim(e_i, e_j) = \alpha \cdot Sim_{COS}(e_i, e_j) - (1-\alpha) \cdot Sim_{Euc}(e_i, e_j), e_i, e_j \in E_{Entity} \tag{18}$$

where the cosine similarity is $Sim_{COS}(e_i, e_j) = \overrightarrow{e_i} \cdot \overrightarrow{e_j} / (\overrightarrow{e_i} \cdot \overrightarrow{e_j})$, which measures the angular similarity between the vectors to determine whether two entities are semantically similar. Further, the Euclidean distance is $Sim_{Euc}(e_i, e_j) = \overrightarrow{e_i} - \overrightarrow{e_j}_2$, which quantifies the dissimilarity between entities based on the straight-line distance between their embeddings. The $\alpha \in [0,1]$ is a hyperparameter to balance directional similarity and distance-based difference. When the computed similarity score exceeds a predefined threshold τ, the two entities are considered semantically aligned and a connection is established between their corresponding nodes. This process facilitates the completion of the MPDKG construction.

4.3. Construction of the DRL-based dynamic scheduling module

4.3.1. Scheduling feature extraction based on SHGAT

In the MPDKG, the semantic features embedded in operation knowledge nodes, machine knowledge nodes, and worker knowledge nodes, denoted as $g_i^{\mathscr{P}} \in \mathbb{R}^{d_{\mathscr{P}}}$ where $\mathscr{P} \in \{o,m,w\}$, together with the semantic features of machine-operation and worker-operation relationships, denoted as $u_{ij}^{\mathscr{E}} \in \mathbb{R}^{d_{\mathscr{E}}}$ where $\mathscr{E} \in \{r^{mo}, r^{wo}\}$, constitute the core scheduling knowledge that encodes scheduling status and guides agent training. The detailed specifications of semantic features are provided in Appendix Table 1. To effectively extract scheduling knowledge from the MPDKG, graph embedding must account for both the semantic features of knowledge nodes and the structural characteristics of the graph. To this end, this paper proposes the SHGAT framework for multi-source feature fusion. As illustrated in Fig. 6, SHGAT performs both relation-aware and node-aware semantic aggregation, enabling the integration of heterogeneous relations and neighbour nodes. This approach significantly enhances the semantic expressiveness and representation capability of the resulting node embeddings.

4.3.1.1. Relation-aware semantic aggregation. Relation-aware semantic aggregation aims to integrate relation-specific semantic information from heterogeneous relations into the central node, rather than treating all relations equally. For a knowledge node i with semantic features, $g_i^{\mathscr{P}} \in \mathbb{R}^{d_{\mathscr{P}}}$ where $\mathscr{P} \in \{o,m,w\}$, the aggregation is computed as:

$$gr_i^{\mathscr{P}(1)} = \sigma\left(\boldsymbol{W}_{\mathscr{P}}^{r(1)} g_i^{\mathscr{P}} + \sum_{\mathscr{E}} \sum_{j \in N_i^{\mathscr{E}}} \boldsymbol{W}_{\mathscr{E}}^{r(1)} u_{ij}^{\mathscr{E}}\right), \mathscr{P} \in \{o,m,w\}, \mathscr{E} \in \{r^{mo}, r^{wo}\} \tag{19}$$

where σ denotes the activation function. $\mathscr{P}$ and $\mathscr{E}$ denote the node type and relation type, respectively. Specifically, $j \in N_i^{\mathscr{E}}$ refers to the neighbouring knowledge nodes j that are connected to node i via a semantic relation $u_{ij}^{\mathscr{E}}$. Transformation matrices $\boldsymbol{W}_{\mathscr{P}}^{r(1)} \in \mathbb{R}^{d \times d_{\mathscr{P}}}$ and $\boldsymbol{W}_{\mathscr{E}}^{r(1)} \in \mathbb{R}^{d \times d_{\mathscr{E}}}$ project features into a unified latent space $\mathbb{R}^d$. Subsequently, the aggregated result will be passed through a layer normalisation function *LayerNorm* and an activation layer:

$$gr_i^{\mathscr{P}(1)} = ReLU\left(LayerNorm\left(gr_i^{\mathscr{P}(1)}\right)\right), \mathscr{P} \in \{o,m,w\} \tag{20}$$

where *ReLU* is a rectified linear unit activation function, yielding the

relation-aware aggregation vector $gr_i^{\mathscr{P}(1)} \in \mathbb{R}^d$.

4.3.1.2. Node -aware semantic aggregation. To account for the varying importance of neighbouring nodes during information propagation, a node-aware semantic aggregation mechanism is introduced using an attention mechanism. For any node i, the attention mechanism is employed to compute the importance weight $e_{ij}^{\mathscr{P}}$ of each neighbouring node $j \in N_i$, including i itself:

$$e_{ij}^{\mathscr{P}(1)} = LeakyReLU\left(a_{\mathscr{P}}^T\left[\boldsymbol{W}_{\mathscr{P}}^{\boldsymbol{n}(1)} g_i^{\mathscr{P}} \| \boldsymbol{W}_{\mathscr{P}}^{\boldsymbol{n}(1)} g_j^{\mathscr{P}}\right]\right), \mathscr{P}, \mathscr{P} \in \{o, m, w\} \tag{21}$$

where $a_{\mathscr{P}}^T \in \mathbb{R}^{2d}$ is the attention weight vector and $\|$ denotes vector concatenation. $\mathscr{P}$ and $\mathscr{P}$ denote the node type and relation type, respectively. $\boldsymbol{W}_{\mathscr{P}}^{\boldsymbol{n}(1)} \in \mathbb{R}^{d \times d_{\mathscr{P}}}$ and $\boldsymbol{W}_{\mathscr{P}}^{\boldsymbol{n}(1)} \in \mathbb{R}^{d \times d_{\mathscr{P}}}$ are used to align feature dimensions across different node types. Subsequently, the attention coefficients $\alpha_{ij}^{\mathscr{P}(1)}$ are then normalised via *softmax*:

$$\alpha_{ij}^{\mathscr{P}(1)} = \frac{exp\left(e_{ij}^{\mathscr{P}(1)}\right)}{\sum_{y \in N_i} exp\left(e_{iy}^{\mathscr{P}(1)}\right)}, \mathscr{P} \in \{o, m, w\} \tag{22}$$

The node-aware aggregation vector of knowledge node i, denoted as $gn_i^{\mathscr{P}(1)} \in \mathbb{R}^d$, is then derived by aggregating the semantic features g_j of its neighbouring nodes j weighted by the corresponding attention coefficients $\alpha_{ij}^{\mathscr{P}}$, followed by a non-linear activation function σ. The aggregation process is formulated as:

$$gn_i^{\mathscr{P}(1)} = \sigma\left(\sum_{j \in N_i} \alpha_{ij}^{\mathscr{P}} \boldsymbol{W}_{\mathscr{P}}^{\boldsymbol{n}(1)} g_j^{\mathscr{P}}\right), \mathscr{P}, \mathscr{P} \in \{o, m, w\} \tag{23}$$

The resulting aggregated vector is then passed through layer normalisation, followed by a non-linear activation function, yielding the node-aware aggregation vector $gn_i^{\mathscr{P}(1)} \in \mathbb{R}^d$.:

$$gn_i^{\mathscr{P}(1)} = ReLU\left(LayerNorm\left(gn_i^{\mathscr{P}(1)}\right)\right), \mathscr{P} \in \{o, m, w\} \tag{24}$$

Given the diversity and significance of adjacent knowledge nodes and semantic relationship types within the MPDKG, the SHGAT model enables more accurate and semantically enriched graph embeddings. These enhanced embeddings support more efficient decision-making in the dynamic scheduling module.

4.3.1.3. Feature fusion via stacked SHGAT layers. The final embedding of node i at the first SHGAT layer is derived by summing the outputs of the relation-aware and node-aware semantic aggregation components. Specifically, the embedding at the first SHGAT layer is computed as:

$$g_i^{\mathscr{P}(1)} = gr_i^{\mathscr{P}(1)} + gn_i^{\mathscr{P}(1)}, \mathscr{P} \in \{o, m, w\} \tag{25}$$

By stacking N SHGAT layers with independent parameters, the final embedding $g_i^{\mathscr{P}(N)} \in \mathbb{R}^d$, where $\mathscr{P} \in \{o, m, w\}$, is obtained. Given the temporal dynamics of the MPDKG, these embeddings and their associated semantic features are time-sensitive. To obtain a unified global representation $sv_t \in \mathbb{R}^{3d}$ at time step t, the mean of the final embeddings for each entity type is computed and concatenated as follows:

$$sv_t = \left[\frac{1}{|O|}\sum_{j=1}^{|O|} g_j^{o(N)} \middle\| \frac{1}{|M|}\sum_{k=1}^{|M|} g_k^{m(N)} \middle\| \frac{1}{|W|}\sum_{r=1}^{|W|} g_r^{w(N)}\right] \tag{26}$$

where $|O|$, $|M|$, and $|W|$ denote the number of operation, machine, and worker nodes in the MPDKG, respectively.

4.3.2. Markov decision process formulation for DRCFJSP

Markov Decision Process (MDP) form the mathematical foundation of DRL. Based on the characteristics of the DRCFJSP, a multi-action MDP model $\langle S, A, P, R\rangle$ is constructed, as follows:

State: $S = \{S_o, S_m, S_w\}$ represents the state space. At time step t, s_t includes the local states of the process s_t^o, machine s_t^m, and worker s_t^w. When O_{ij} is the first operation of J_i, its start time is given by $S_{ij}(t) = AT_i$. When O_{ij} is not the first operation of J_i, and if the previous operation O_{ij-1} is known to be processed by W_r on M_k, then $S_{ij}(t) = S_{ij-1}(t) + p_{ij-1kr}$; otherwise, $S_{ij}(t) = S_{ij-1}(t) + \overline{p}_{ij-1}$, where $\overline{p}_{ij-1}$ is the estimated processing time of the unscheduled operation O_{ij-1}. $\overline{p}_{ij-1}$ is calculated as follows:

$$\overline{p}_{ij-1} = \frac{1}{\left|M_{ij-1}\right| \times \left|W_{ij-1}\right|}\sum_{M_k \in M_{ij-1}}\sum_{W_r \in W_{ij-1}} p_{ij-1kr} \tag{27}$$

Action: $A = \{A_o, A_m, A_w\}$ represents the action space. The process action space A_o consists of the set of unscheduled processes; the machine action space A_m consists of the set of available machines; and the worker action space A_w consists of the set of available workers. At time step t, the agent samples an action $a_t \in A_t$, where $a_t = \{a_t^o, a_t^m, a_t^w\}$. Specifically, $a_t^o \in A_t^o$, $a_t^m \in A_t^m$, and $a_t^w \in A_t^w$.

Transition: $P(s_{t+1}|s_t, a_t)$ represents the state transition probability set. At time step t, the agent chooses an action a_t based on state s_t and interacts with the environment, to transition to a new state s_{t+1}.

Reward: The reward function R is closely related to the objective function. The instantaneous rewards corresponding to the two optimisation objectives, makespan C_{max} and total cost L_{cost}, are defined as r_1 and r_2, respectively. r_1 is calculated as follows:

$$r_1 = C_{max}(s_t) - C_{max}(s_{t+1}) \tag{28}$$

where $C_{max}(s_t)$ represents the maximum completion time at time step t. The calculation for r_2 is as follows:

$$r_2 = ratio(t) - ratio(t+1) \tag{29}$$

where $ratio(t) = TP(t)/L_c(t)$ is the ratio between the total processing time of workers and machines $TP(t) = \sum_{i=1}^{n}\sum_{j=1}^{n_i}\sum_{k=1}^{m}\sum_{r=1}^{l} x_{ijkr} \times p_{ijkr} \times 2$ and the total cost $L_{cost}(t)$. Using the weighted summation method [73], the two objectives are combined into a single objective by constructing a weight vector $\omega_i = (\omega_1, \omega_2)$, where$\omega_1, \omega_2 \geq 0$ and $\omega_1 + \omega_2 = 1$. The weighted reward is $R_t = \omega_1 r_1 + \omega_1 r_2$.

4.3.3. MA-PPO algorithm

To solve the DRCFJSP with multiple action spaces, an MA-PPO algorithm is proposed. This extends the PPO [50] from a single to multiple action spaces, applied to train the process, machine, and worker actors. The embedding vector $h_i^{o(N)}$ of the process action a_t^o corresponding to the knowledge node i is concatenated with the global vector sv_t and processed using the multi-layer perceptron (MLP) MLP_{θ_o} to obtain the priority coefficient of a_t^o in state s_t^o:

$$P_t^{o,i}\left(a_t^o, s_t^o\right) = MLP_{\theta_o}\left[h_i^{o(N)} \| sv_t\right] \tag{30}$$

Subsequently, the *softmax* function is used to normalise $P_t^{o,i}\left(a_t^o, s_t^o\right)$ and obtain the process actor:

$$\pi_{\theta_o}\left(a_t^o | s_t^o\right) = \frac{exp\left(P_t^{o,ij}\left(a_t^o, s_t^o\right)\right)}{\sum_{a_t^o \in A_t^o} exp\left(P_t^{o,ij}\left(a_t^o, s_t^o\right)\right)} \tag{31}$$

Similarly, the embedding vector $h_k^{m(N)}$ for machine action a_t^m and the embedding vector $h_r^{w(N)}$ for worker action a_t^w are concatenated with the global vector sv_t and processed using MLP_{θ_m} and MLP_{θ_w}, respectively, to obtain the corresponding priority coefficients $P_t^{m,k}\left(a_t^m, s_t^m\right)$ and $P_t^{w,r}\left(a_t^w, s_t^w\right)$. Specifically, the three networks share only the SHGAT, whereas their actor network parameters are independent. The machine actor $\pi_{\theta_m}\left(a_t^m | s_t^m\right)$ and worker actor$\pi_{\theta_w}\left(a_t^w | s_t^w\right)$ are obtained after normalisation. During training, the agent randomly samples based on the multi-actor $\pi_{\theta_{\mathscr{P}}}\left(a_t^{\mathscr{P}} | s_t^{\mathscr{P}}\right)(\mathscr{P} \in \{o, m, w\})$ to fully explore the environment.

The critic $v_\phi(s_t)$ shares the same SHGAT model as the multi-actor

Algorithm 1
MA-PPO for DRCFJSP.

Input: Training steps E_t, network update steps E_u, SHGAT model, target mulit-actor network $\pi_{\theta_{\mathscr{P}}}(\mathscr{P} \in \{o,m,w\})$and critic network v_ϕ with initial parameters ω, $\theta_{\mathscr{P}}(\mathscr{P} \in \{o,m,w\})$ and ϕ
Initialize: $\theta_{\mathscr{P}}^{old} \leftarrow \theta_{\mathscr{P}}(\mathscr{P} \in \{o,m,w\})$
Sample a batch of B DRCFJSP instances
for $e_t = 1, \cdots, E_t$ **do**
for $t = 0, 1, \cdots$ **do**
Extract embedding using SHGAT;
Sample $a_t^{\mathscr{P}}(\mathscr{P} \in \{o,m,w\})$ according to $\pi_{\theta_{\mathscr{P}}}(\mathscr{P} \in \{o,m,w\})$;
Observe reward r_t and next state s_{t+1};
Compute the generalized advantage estimates $\widehat{A_t}$;
if s_t is terminal **then**
break;
end
end
for $e_u = 1, \cdots, E_u$ **do**
Compute the probability ratio $\delta_t^{\mathscr{P}}(\theta_{\mathscr{P}})(\mathscr{P} \in \{o,m,w\})$;
Compute the $L_{CLIP}(\theta_{\mathscr{P}})(\mathscr{P} \in \{o,m,w\})$;
Compute the $L_E(\theta_{\mathscr{P}})(\mathscr{P} \in \{o,m,w\})$;
Compute the actor Loss: $L(\theta_{\mathscr{P}})(\mathscr{P} \in \{o,m,w\})$;
Compute the critic Loss: $L_{MSE}(\phi)$;
Update the parameters ω, $\theta_{\mathscr{P}}(\mathscr{P} \in \{o,m,w\})$ and ϕ by a gradient;
end
Update behavior actor network by $\theta_{old} \leftarrow \theta$;
end
Output: Trained parameter sets ω, $\theta_{\mathscr{P}}(\mathscr{P} \in \{o,m,w\})$ and ϕ of all networks.

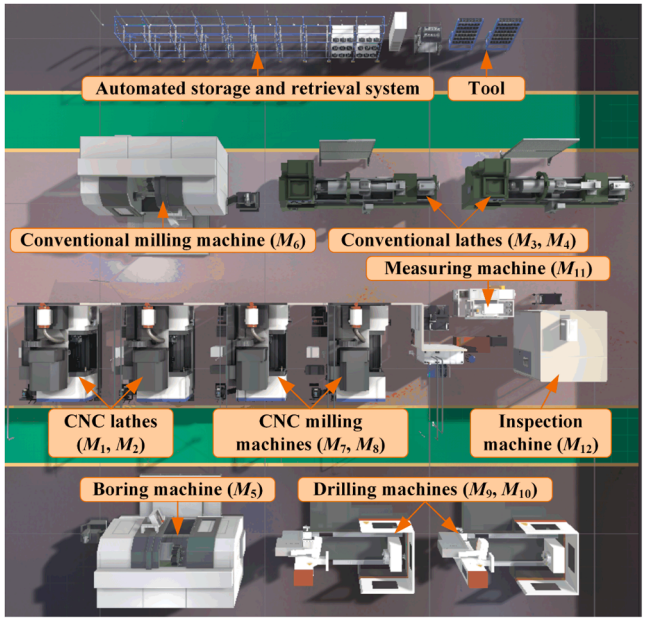


Fig. 7. Layout of the aerospace manufacturing workshop.

architecture. It receives the global scheduling vector sv_t as input through an MLP_ϕ: $v_\phi(s_t) = MLP_\phi(sv_t)$. The critic evaluates the state value of the entire scheduling process and guides the optimisation of each actor's scheduling behaviour through the sv_t. It learns to approximate the state value function $v_\phi(s_t)$, and compute the advantage function is computed as:

$$A_t = \sum_{t'=t}^{T} \gamma^{t'} r_{t'} - v_\phi(s_t) \tag{32}$$

where γ is the discount factor, t' is the current timestep, and $T \ll |O|$. In each training iteration, the multi-actor interacts with the environment to collect experience tuples and updates both the multi-actor $\pi_{\theta_{\mathscr{P}}}(a_t^{\mathscr{P}}|s_t^{\mathscr{P}})(\mathscr{P} \in \{o,m,w\})$ and the critic $v_\phi(s_t)$. Specifically, the multi-actor is trained to maximise the following objective:

$$L_{LOSS}(\theta_{\mathscr{P}}) = c_p L_{CLIP}(\theta_{\mathscr{P}}) + c_e L_E(\theta_{\mathscr{P}}), \mathscr{P} \in \{o,m,w\} \tag{33}$$

where $c_p = 1$ and $c_e = 0.01$ [50] are clip and entropy coefficient hyperparameters, $L_{CLIP}(\theta_{\mathscr{P}})$ is the clipped surrogate objective and $L_E(\theta_{\mathscr{P}})$ is the entropy objective. They are calculated respectively as follows:

$$L_{CLIP}(\theta_{\mathscr{P}}) = E_t[min\{\delta_t(\theta_{\mathscr{P}})\widehat{A}_t, clip(\delta_t(\theta_{\mathscr{P}}), 1-\epsilon, 1+\epsilon)\widehat{A}_t\}], \mathscr{P} \in \{o,m,w\} \tag{34}$$

$$L_E(\theta_{\mathscr{P}}) = E_t\left[Entropy\left(\pi_{\theta_{\mathscr{P}}}\left(a_t^{\mathscr{P}}|s_t^{\mathscr{P}}\right)\right)\right], \mathscr{P} \in \{o,m,w\} \tag{35}$$

where $\delta_t(\theta_{\mathscr{P}}) = \pi_{\theta_{\mathscr{P}}}\left(a_t^{\mathscr{P}}|s_t^{\mathscr{P}}\right)/\pi_{\theta_{\mathscr{P}}^{old}}\left(a_t^{\mathscr{P}}|s_t^{\mathscr{P}}\right)$ is the probability ratio between the target actor $\pi_{\theta_{\mathscr{P}}}(\mathscr{P} \in \{o,m,w\})$ and the behavior actor $\pi_{\theta_{\mathscr{P}}^{old}}(\mathscr{P} \in \{o,m,w\})$ used for sampling, and $\epsilon \in (0,1)$ is the clipping parameter. The critic $v_\phi(s_t)$ is updated by minimising the mean squared error:

$$L_{MSE}(\phi) = E_t\left[MSE\left(r_t, v_\phi(s_t)\right)\right] \tag{36}$$

where r_t is the immediate reward at time t. The pseudocode of the MA-PPO algorithm is shown in Algorithm 1.

5. Experimental

This section presents a series of experiments to validate the effectiveness of the proposed method for solving the DRCFJSP in dynamic manufacturing environments. First, the experimental setup is described, followed by detailed procedures for MDPKG construction and MA-PPO training. Finally, the proposed approach is evaluated through comparative analysis with baseline methods, including PPO [50] and DDQN [51] methods.

5.1. Experimental design

This section presents numerical experiments in the context of an aerospace manufacturing workshop, with the layout illustrated in Fig. 7. The workshop is equipped with 12 machines and 8 workers, and is capable of producing 6 types of workpieces that collectively involve 10 different types of operations. Each machine can perform at least one type

Table 3
Values of manufacturing environment configuration parameters.

Parameters	Value
Number of initial jobs n	$\{10, 20, 30\}$
Number of new arrival jobs n_{new}	$\{10, 20\}$
Average value of the new job arrival interval β_p	$\{20, 100\}$
Worker skill level parameters PL_{ijr}	$\{p_I = 0.8, p_{II} = 1.0, p_{III} = 1.2\}$
Machine cost per unit time MC_{ijk}	The numerical information of these specific entities (machines/workers) is referenced from [30] and stored in the semantic features of the corresponding knowledge nodes in the workshop MPDKG and will be retrieved in Section 5.2 and filled into Tables 7 and 8.
Worker cost per unit time WC_{ijr}	
Machine parameter me_k	
Worker parameter we_r	
Fatigue impact on processing time δ^k	
Preset processing time p_{ijk}	

Table 4
Hyperparameter information.

Hyperparameter	Value
Learning rate lr	2×10^{-4}
Clip coefficient c_e	1
Entropy coefficient c_p	0.01
Clip coefficient ϵ	0.2
Batch of instances B	30
Network update steps E_u	4
Discount factor γ	0.95

of operation, and each worker is skilled in at least three different operations.

To simulate realistic production scenarios, randomised orders are generated in the test instances by selecting from the six available workpiece types. In each instance, an initial batch of orders is released at the beginning of the scheduling process, whereas new orders arrive randomly during production to emulate dynamic conditions. To evaluate the generalisability and robustness of the proposed method across diverse manufacturing environments, the configuration parameters used in the experiments are provided in Table 3.

The MA-PPO agent was trained on a PC equipped with a 13th Gen Intel(R) Core(TM) i7-13700F CPU @ 2.10 GHz and 32 GB of RAM, using PyTorch version 2.1.2. Following the training protocol proposed by Song et al. [52], the number of training iterations was set to $E_t = 1000$. The number of policy optimisation updates was set to $E_u = 10$, with a discount factor $\gamma = 1.0$. The SHGAT module consists of $N = 2$ layers. Both $MLP_{\theta_{\mathscr{P}}}(\mathscr{P} \in \{o, m, w\})$ and MLP_{ϕ}, are constructed with two hidden layers, each comprising 32 units, and employed the *tanh* activation function. Network parameters were updated using the Adam optimizer is used for network parameter updates. The remaining key hyperparameterares shown in the Table 4.

Given the interdependence and partial conflict between the two optimization objectives, the training process yields a set of Pareto-optimal solutions. The final scheduling decision is selected from this solution set using the TOPSIS [53]. TOPSIS is a multi-criteria decision analysis method that evaluates alternatives by calculating their relative distances to the ideal and negative-ideal solutions, selecting the solution closest to the ideal. To evaluate the performance of the algorithms, three commonly used metrics are adopted: GD [54], Spread [55], and IGD [56]. Lower values of these metrics indicate better convergence and diversity of the solution set.

$$GD(A, X^*) = \frac{\sqrt{\sum_{i=1}^{|A|} D_i^2(A, X^*)}}{|A|} \tag{37}$$

where A is the optimal Pareto front obtained by the evaluated algorithm, X^* is the true Pareto front (True-PF), and $D_i(A, X^*)$ is the minimum Euclidean distance between solution i in A and all solutions in X^*.

$$Spread = \frac{\sum_{j=1}^{n_0} D_j^e(A, X^*) + \sum_{i=1}^{|A|} |D_i(A, A) - \overline{D}_i(A, A)|}{\sum_{j=1}^{n_0} D_j^e(A, X^*) + |A| \cdot \overline{D}_i(A, A)} \tag{38}$$

where $D_i(A, A)$ is the Euclidean distance between solution i in A and its

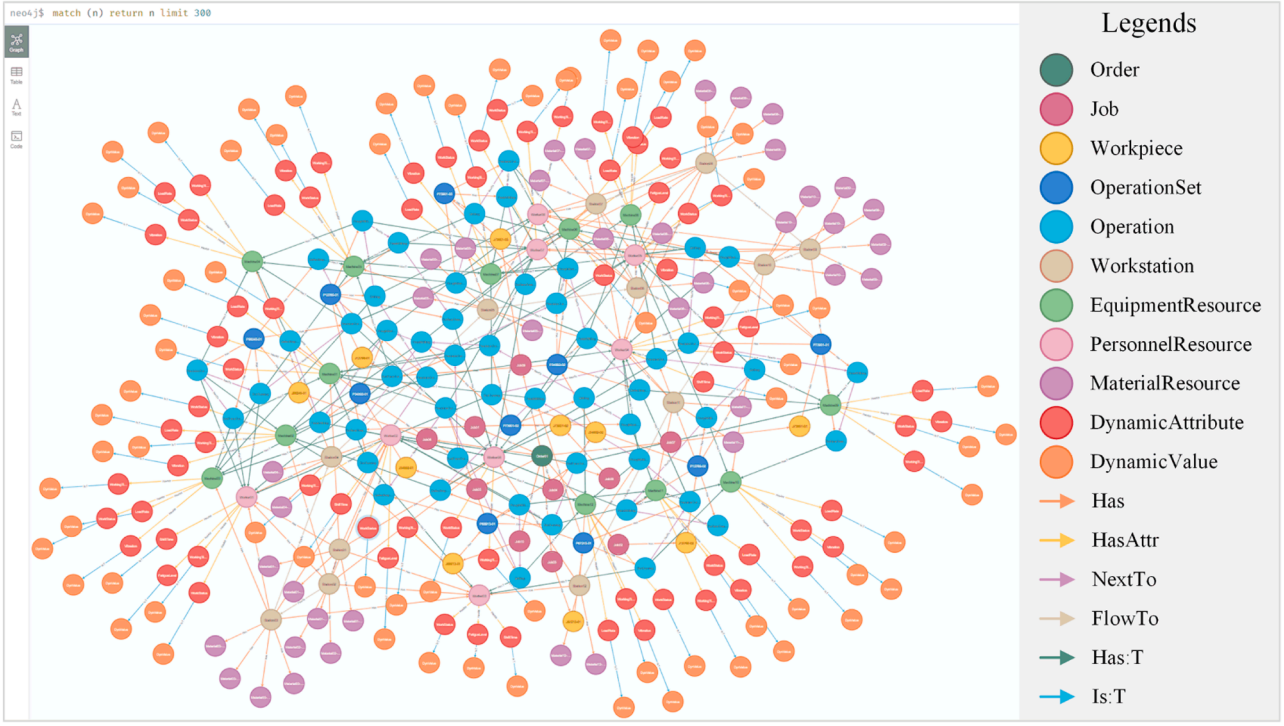


Fig. 8. A localized view of the constructed MPDKG.

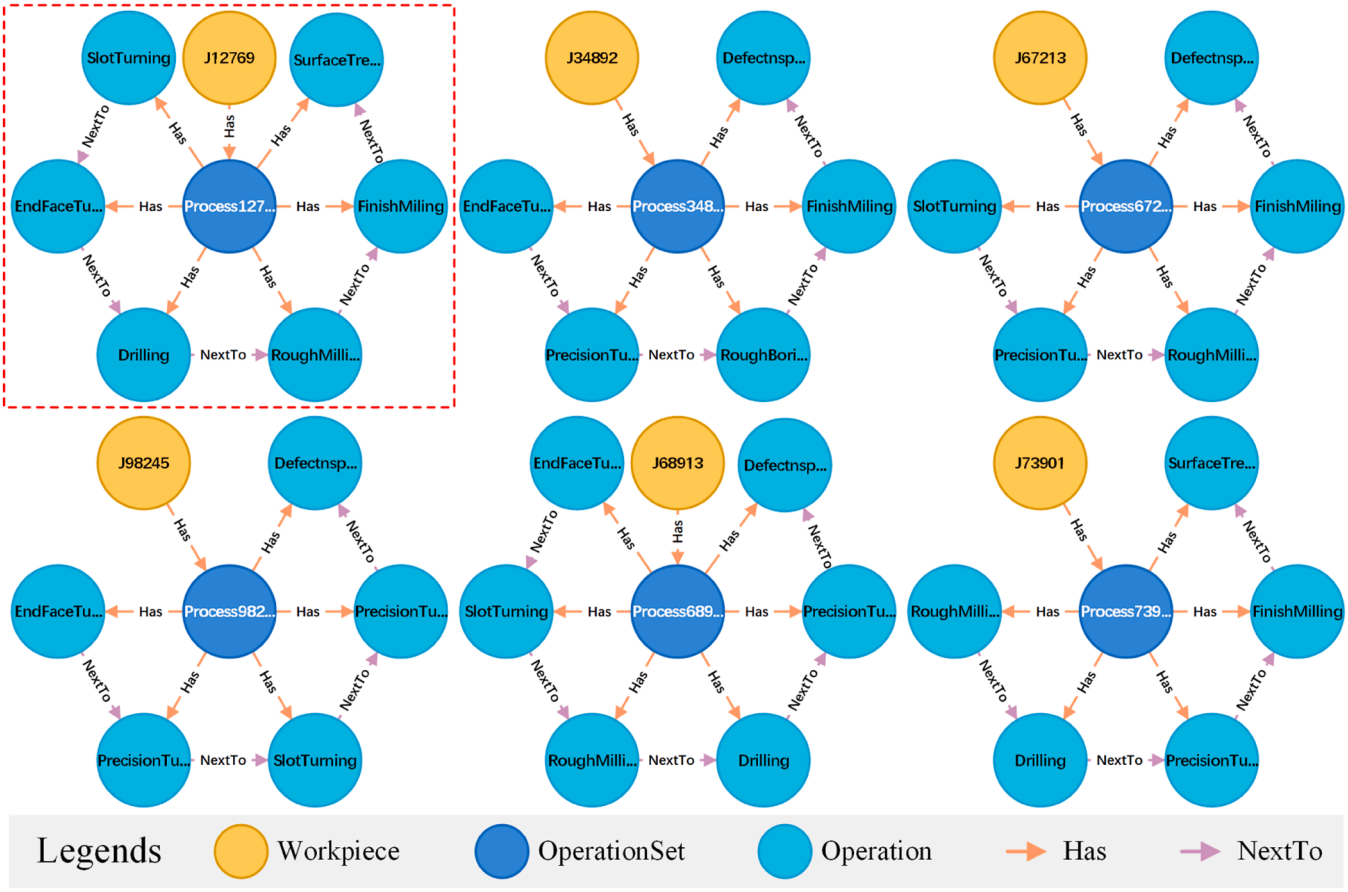


Fig. 9. Jobs information obtained via MPDKG query.

Table 5
Workpieces process flow information.

Name	Operation 1	Operation 2	Operation 3	Operation 4	Operation 5	Operation 6
J12769	slot turning	precision turning-2	drilling	rough milling	finish milling	surface treatment
J34892	end face turning	precision turning-1	rough boring	finish milling	defect inspection	-
J67213	slot turning	precision turning-1	rough milling	finish milling	defect inspection	-
J98245	end face turning	precision turning-2	slot turning	precision turning-1	defect inspection	-
J73901	rough boring	drilling	precision turning-1	finish milling	surface treatment	-
J68913	end face turning	slot turning	rough milling	drilling	precision turning-2	defect inspection

Table 6
Operations and alternative machines and workers of the case.

Operations	Machines	p_{ijk}	Workers	PL_{ijr}
slot turning	M_1, M_2, M_3, M_4	[7,8,10,16]	W_1, W_2, W_3	II, II, III
end face turning	M_1, M_2, M_3, M_4	[16,19,18,12]	W_1, W_2, W_3	II, III, II
precision turning-1	M_1, M_2	[10,13]	W_1, W_2, W_8	III, I, II
precision turning-2	M_1, M_2	[7,6]	W_1, W_2, W_8	II, I, III
rough boring	M_5	[14]	W_4, W_6	I, III
rough milling	M_6, M_7, M_8	[13,9,10]	W_4, W_5, W_6	II, III, I
finish milling	M_7, M_8	[15,14]	W_4, W_5, W_7	III, II, I
drilling	M_9, M_{10}	[14,15]	W_5, W_6, W_7	II, I, III
surface treatment	M_{11}	[17]	W_4, W_7	II, I
defect inspection	M_{12}	[16]	W_3, W_8	I, III

nearest neighbour in A, $\overline{D}_i(A,A)$ is the average of such distances, and $D_j^e(A,X^*)$ represents the distance between the extreme solution of A and the boundary solution of X^* in the jth objective. n_0 denotes the number of objectives.

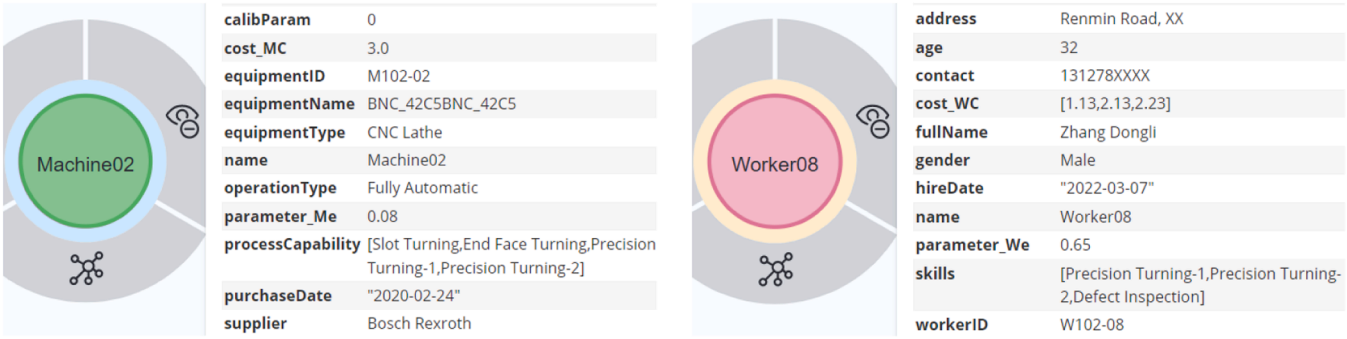
$$IGD(X^*,A) = \frac{\sum_{i=1}^{|X^*|} D_i(X^*,A)}{|X^*|} \tag{39}$$

where $D_i(X^*,A)$ is the minimum Euclidean distance from solution i in X^* to all solutions in A. Because the True-PF is unknown in dynamic scheduling problems, the Pareto Front formed by the solutions of the comparative algorithms is regarded as the True-PF.

5.2. Generation of the MPDKG for aerospace manufacturing workshop

Based on the proposed MPDKG construction framework, an MPDKG for the aerospace manufacturing workshop was constructed and deployed using Neo4j for efficient storage and management. The resulting graph consists of 2479 knowledge nodes and 3736 semantic relationships, effectively capturing the complex interactions among manufacturing resources, processes, and tasks. A partial visualisation of the constructed MPDKG is presented in Fig. 8.

The process flow information for six representative workpieces was extracted from the MPDKG, as illustrated in Fig. 9. For instance, the



(a) Machine node semantic attributes (b) Worker node semantic attributes

Fig. 10. Semantic attributes of candidate machine and worker nodes.

Table 7
Machine-related information.

No.	me_k	MC_{ijk}	δ^k	No.	me_k	MC_{ijk}	δ^k
M_1	0.05	3.2	0	M_7	0.16	2.9	0
M_2	0.08	3.0	0	M_8	0.14	3.0	0
M_3	0.18	2.5	$0.3/ln2$	M_9	0.12	2.6	$0.3/ln2$
M_4	0.10	2.8	$0.3/ln2$	M_{10}	0.18	2.7	$0.3/ln2$
M_5	0.12	2.6	$0.7/ln2$	M_{11}	0.16	2.9	$0.7/ln2$
M_6	0.18	2.7	$0.3/ln2$	M_{12}	0.14	3.0	$0.7/ln2$

Table 8
Worker-related information.

No.	we_r	WC_{ijr}			No.	we_r	WC_{ijr}		
		I	II	III	I		I	II	III
W_1	0.75	1.91	2.13	2.21	W_5	0.70	-	2.16	2.23
W_2	0.50	1.45	2.11	2.23	W_6	0.45	2.03	-	2.21
W_3	0.60	1.6	2.12	2.19	W_7	0.80	1.64	2.11	2.18
W_4	0.55	1.16	2.10	2.21	W_8	0.65	1.13	2.18	2.23

process sequence for job *J12769* is: slot turning → precision turning-2 → drilling → rough milling → finish milling → surface treatment. This extracted information is organised and listed in Table 5. Additional queries were performed to retrieve the candidate machines and workers associated with each process step. Although the query content varies according to the information sought, the underlying query structure remains consistent with that used for process flow extraction. The corresponding results are listed in Table 6.

The semantic features required for SHGAT-based graph embedding are stored within the attribute fields of corresponding nodes and relationships in the MPDKG. Taking machine and worker knowledge nodes as examples, their semantic features are illustrated in Fig. 10 (a) and (b), respectively. Fig. 10 (a) presents the 11 static attributes of

Table 10
Benchmark rules.

No.	Operation rule	Worker rule
R1	SPT	LWC
R2	MWKR	
R3	MOR	
R4	SPT	HPL
R5	MWKR	
R6	MOR	

Table 9
Comparison of ablation study.

n	β_p	n_{new}	Object	Ours	II	III	I	Ours	II	III	I
				Average				Compromise Solutions			
10	20	10	C_{max}	**181.14**	184.39	184.70	185.16	**185.97**	184.38	179.12	183.08
			L_{cost}	**4554.98**	4611.24	4585.13	4591.85	**4417.83**	4537.60	4571.14	4542.59
		20	C_{max}	**198.69**	203.74	208.7	206.73	**199.69**	202.09	202.29	206.49
			L_{cost}	**5084.58**	5128.19	5110.99	5122.46	**4983.5**	5081.31	5087.7	5077.97
	200	10	C_{max}	**169.32**	172.03	174.7	173.7	**159.11**	170.93	166.72	166.68
			L_{cost}	**4257.3**	4270.48	4283.29	4287.69	**4227.54**	4236.1	4281.43	4288.81
		20	C_{max}	**220.11**	225.13	225.85	227.14	**215.15**	219.72	225.34	224.14
			L_{cost}	**6586.27**	6669.98	6684.11	6700.26	**6476.68**	6678.02	6627.67	6654.43
20	20	10	C_{max}	**237.99**	243.16	244.54	248	**236.83**	242.82	238.86	244.29
			L_{cost}	**6874.09**	7007.37	7015.43	7031.87	**6654.17**	6914.16	6982.99	6979.59
		20	C_{max}	**243.53**	242.37	243.57	244.86	**234.37**	240.98	241.02	241.47
			L_{cost}	**8343.34**	8473.5	8502.84	8491.73	**8346.38**	8462.49	8486.43	8503.97
	200	10	C_{max}	**212.82**	216.07	218.32	218.27	**209.77**	216.47	220.26	220.2
			L_{cost}	**6849.78**	6918.79	6935.57	6944.18	**6684.77**	6843.21	6807.9	6829.07
		20	C_{max}	**282.17**	287.02	290.22	288.64	**278.05**	278.05	278.05	278.05
			L_{cost}	**9971.48**	10077.25	10096.04	10108.35	**9847.43**	9847.43	9847.43	9847.43
30	20	10	C_{max}	**251.45**	258.27	262.91	260.89	**248.26**	264.01	250.29	246.77
			L_{cost}	**9028.73**	9140.53	9162.16	9185.02	**8736.17**	8832.47	9268.67	9381.11
		20	C_{max}	**252.29**	258.6	258.01	259.63	**245.23**	249.84	256.76	251.45
			L_{cost}	**9499.75**	9533.66	9536.17	9556.53	**9651.46**	9708.34	9464.11	9693.04
	200	10	C_{max}	**211.72**	219.31	218.79	218.56	**211.88**	222.55	219.26	223.17
			L_{cost}	**8377.57**	8465.55	8478.79	8487.09	**7878.41**	8244.67	8321.01	8247.36
		20	C_{max}	**270.52**	272.99	276.21	275.51	**266.79**	270.48	273.31	272.41
			L_{cost}	**10107.88**	10272.26	10272.33	10298.79	**9906.18**	10144.84	10112.53	10148.15

Table 11
Pareto optimal solution *GD*, *IGD* and *Spread* values.

n	β_p	n_{new}	GD			Spread			IGD		
			Our	PPO	DDQN	Our	PPO	DDQN	Our	PPO	DDQN
10	20	10	**0.0191**	0.0220	0.0309	0.6246	0.6742	**0.6004**	**0.0142**	0.0226	0.0270
		20	0.0553	**0.0874**	0.1034	**0.5456**	0.5892	0.8585	**0.0525**	0.0832	0.1484
	200	10	**0.0449**	0.0532	0.1003	**0.5993**	0.7398	0.9387	**0.0914**	0.1052	0.1161
		20	0.1037	**0.0946**	0.0948	**0.5509**	0.6809	0.7809	**0.0586**	0.1049	0.0593
20	20	10	**0.0317**	0.0469	0.0712	**0.0311**	0.0412	0.0740	0.8488	0.6612	**0.8409**
		20	**0.0452**	0.0480	0.0911	0.0592	**0.0513**	0.0812	**0.7897**	0.7755	0.9225
	200	10	**0.0342**	0.0511	0.0918	0.0297	**0.0283**	0.0799	**0.8003**	0.8010	0.9199
		20	**0.0650**	0.0843	0.1107	**0.0689**	0.0714	0.1120	**0.7107**	0.7563	0.7201
30	20	10	**0.0395**	0.0544	0.0708	**0.0450**	0.1001	0.0902	**0.8770**	0.9424	1.0009
		20	**0.0570**	0.0910	0.1317	**0.0491**	0.0954	0.1496	**0.7743**	0.9040	0.9086
	200	10	**0.0414**	0.0493	0.0466	0.0518	**0.0331**	0.0391	1.0828	1.0253	**0.9564**
		20	**0.0453**	0.1364	0.1287	**0.0206**	0.0811	0.0994	**0.7409**	0.8214	0.8728

Table 12
Pareto optimal solution *GD*, *IGD* and *Spread* values with rules.

n	β_p	n_{new}		Ours	Rule1	Rule2	Rule3	Rule4	Rule5	Rule6
10	20	10	*GD*	**0.0190**	0.6331	0.4682	0.8547	0.5459	0.8353	0.5892
			Spread	**0.6246**	1.0000	0.8954	1.0000	1.0000	0.9290	1.0000
			IGD	**0.0142**	0.7340	0.5001	0.8865	0.5884	0.6354	0.5797
		20	*GD*	**0.0376**	0.8065	0.2414	0.1595	0.3698	0.2259	0.0553
			Spread	**0.5456**	1.0000	0.8738	0.7391	0.7852	0.8115	0.7382
			IGD	**0.0525**	0.9477	0.3253	0.2078	0.4280	0.1309	0.1453
	200	10	*GD*	**0.0490**	0.4978	0.5756	0.6527	0.6118	0.7544	0.6275
			Spread	**0.9053**	1.0000	1.0000	0.9565	0.9687	0.9730	0.9820
			IGD	**0.0498**	0.6676	0.7226	0.7836	0.7337	0.7382	0.7524
		20	*GD*	**0.1662**	0.6233	0.4448	0.3401	0.4731	0.2074	0.2770
			Spread	**0.7693**	1.0000	0.9719	1.0000	0.9152	0.8896	1.0000
			IGD	**0.0899**	0.8816	0.6707	0.5825	0.6763	0.5562	0.5614
20	20	10	*GD*	**0.0230**	0.5358	0.5683	0.6794	0.6057	0.7690	0.6729
			Spread	**0.9801**	1.0000	0.9819	0.9948	0.9884	1.0000	0.9892
			IGD	**0.0225**	0.7239	0.7251	0.8216	0.7538	0.7761	0.7865
		20	*GD*	**0.0178**	1.1458	1.1639	1.0737	1.1390	0.9563	1.0457
			Spread	**0.8420**	1.0000	0.9848	0.9964	0.9901	1.0000	0.9934
			IGD	**0.0582**	1.1410	1.1327	1.0065	1.1111	0.9174	0.9139
	200	10	*GD*	**0.0472**	0.9964	0.9864	0.8729	0.9703	0.6187	0.7188
			Spread	**0.7266**	0.9915	0.9772	0.9870	0.9863	0.9941	0.9912
			IGD	**0.0582**	1.1410	1.1327	1.0065	1.1111	0.9174	0.9139
		20	*GD*	**0.0779**	0.9130	0.8076	0.5951	0.7798	0.4875	0.5747
			Spread	**0.6869**	1.0000	0.9917	1.0000	0.9851	0.9836	0.9947
			IGD	**0.0880**	1.0571	0.9013	0.7432	0.8696	0.7222	0.6947
30	20	10	*GD*	**0.0279**	1.1651	1.2124	1.0899	1.1981	0.9454	1.0269
			Spread	**0.6869**	1.0006	0.9917	1.0000	0.9851	0.9836	0.9947
			IGD	**0.0178**	1.2315	1.2759	1.1578	1.2569	1.0984	1.0981
		20	*GD*	**0.0475**	0.6256	0.3434	0.1589	0.2280	0.0554	0.0957
			Spread	**0.6430**	1.0000	0.9268	0.8799	0.9425	1.0000	0.9131
			IGD	**0.0446**	0.8481	0.4881	0.3761	0.4638	0.2370	0.2128
	200	10	*GD*	**0.0356**	1.0149	1.0303	0.8059	0.9199	0.7503	0.9074
			Spread	**0.9260**	1.0006	0.9361	0.9319	1.0006	0.9477	1.0000
			IGD	**0.0356**	1.0149	1.0303	0.8059	0.9199	0.9074	0.8377
		20	*GD*	**0.0366**	1.0524	0.9672	0.8887	0.9998	0.7356	0.7976
			Spread	**0.7130**	1.0000	1.0000	0.9946	0.9469	0.9682	0.9766
			IGD	**0.0366**	1.0524	0.9672	0.8887	0.9998	0.7976	0.7960

Machine 02, whereas (b) presents the 11 static attributes of Worker 08. Information not included in Table 5 is stored in the respective attributes of these nodes.

By querying the MPDKG as described, all relevant machine information is populated in Table 7, and worker information in Table 8. Additionally, the fatigue recovery constants are set as $\beta_f = 1.4$ and $\gamma_f = 0.05$.

This exemplifies a portion of the information retrieval process. In practical applications, the information fusion module automatically extracts the semantic features that represent the current system state from the MPDKG, using these as input for the dynamic scheduling module.

5.3. Experimental results and analysis

5.3.1. Ablation study

To systematically evaluate the contribution of key components within the proposed feature extraction module to scheduling performance, a series of ablation experiments were conducted. These experiments focused specifically on the mechanisms of semantic modelling and graph embedding. The primary objective is to validate the effectiveness and necessity of the SHGAT in state feature representation and policy learning for dynamic scheduling. The feature extraction module is divided into two core layers: (1) the semantic structure modelling enabled by the construction of the MPDKG, and (2) the semantic feature fusion, achieved through the SHGAT during the graph embedding phase. To evaluate the individual and combined impact of these layers, four

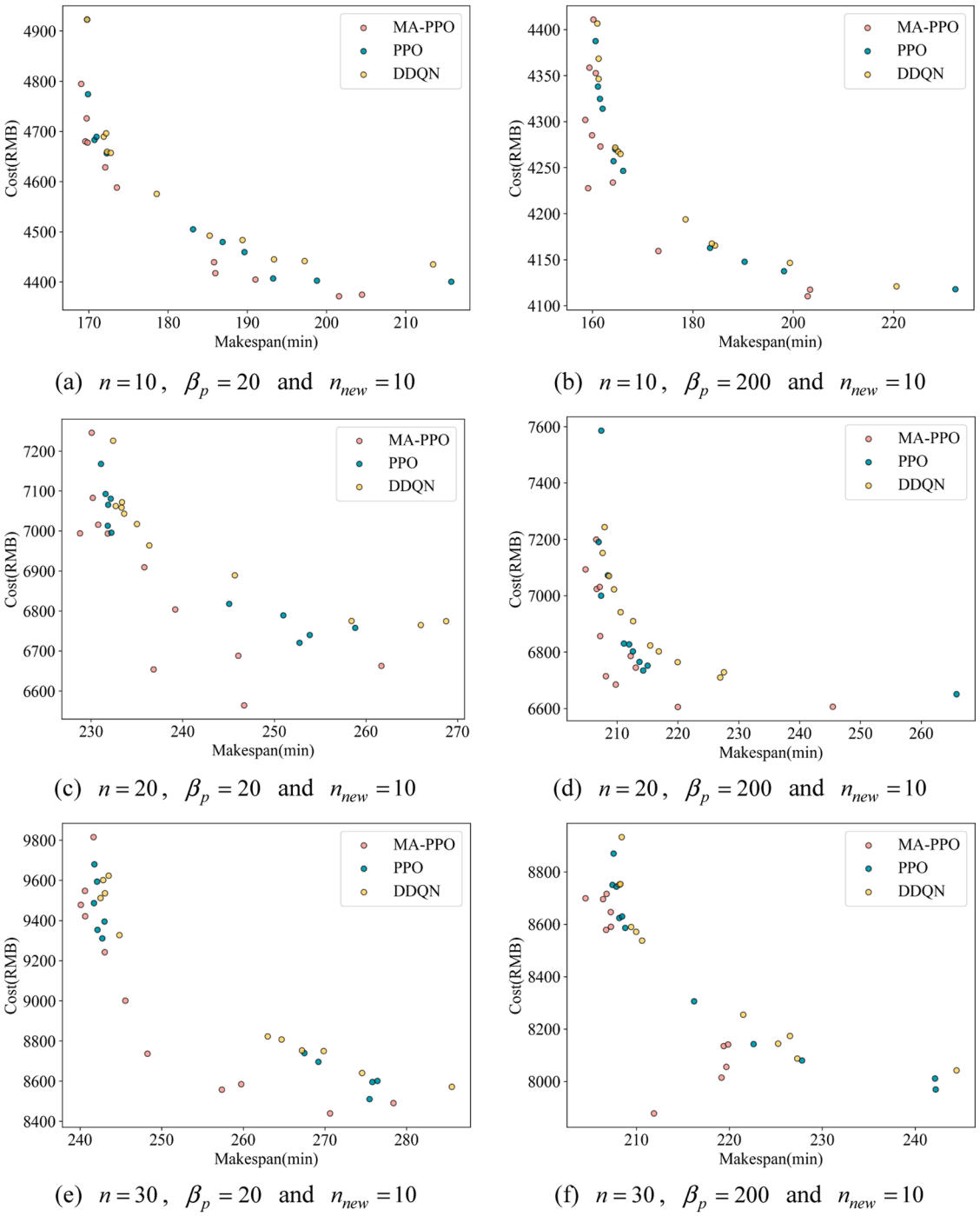


Fig. 11. Pareto front distributions of MA-PPO, PPO, and DDQN under different problem scales.

comparative experiments were designed under consistent scheduling environments and using the same policy algorithm (MA-PPO). These experiments are structured to isolate and examine the effects of each component:

Our model: This is the complete proposed method, incorporating the full SHGAT structure. It serves as the baseline for evaluating the performance impact of each ablated component.

Experiment I: The entire SHGAT module is removed. Instead of performing graph-based aggregation, node attribute features are directly extracted from the MPDKG and concatenated into a one-dimensional feature vector. This vector is fed into the policy network without any structural modelling. This configuration evaluates the necessity and effectiveness of the graph embedding mechanism in fusing semantic features.

Experiment II: The SHGAT module retains edge-type modelling but removes the node-level attention mechanism. Neighbouring nodes are aggregated with equal weight, ignoring their relative importance. This experiment is designed to assess the impact of node-level attention in enhancing the representational quality of graph embeddings.

Experiment III: In this configuration, the relation-specific attention mechanism is disabled. All edge types are treated uniformly, and only node-level attention is retained. This setup aims to analyse the importance of relation-aware modelling and the role of semantic edge types in scheduling policy learning.

All ablation experiments were conducted under identical parameter settings. Performance evaluation focused on two key indicators:

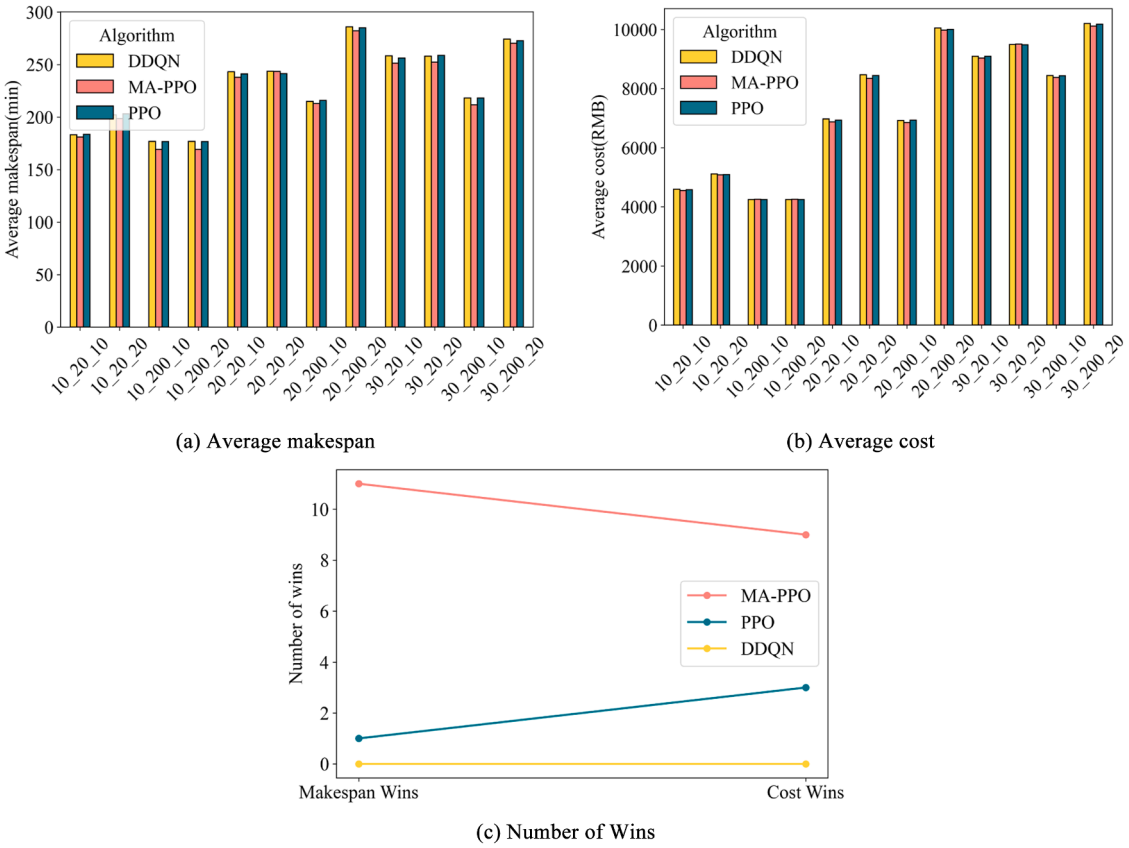


Fig. 12. Comparison of average makespan and average cost across different cases.

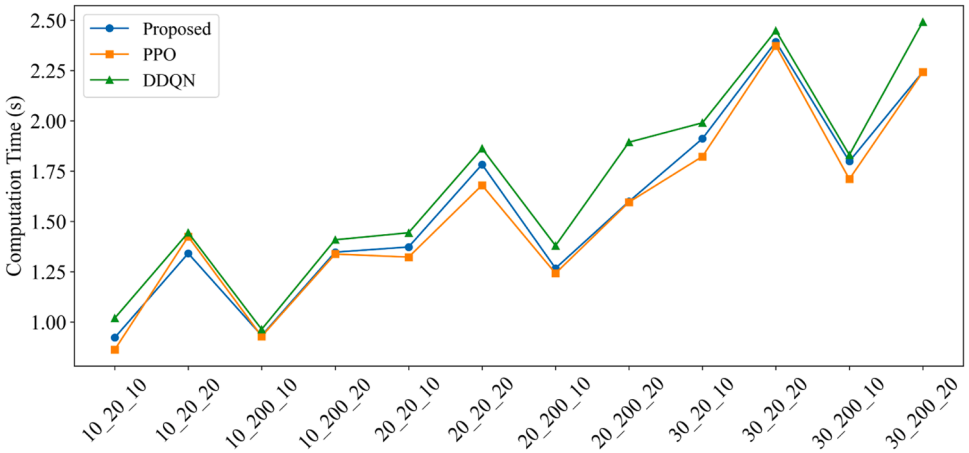


Fig. 13. Comparison of method execution time across different instances.

makespan and total cost, measured for both the compromise solution (selected via TOPSIS) and the average performance across the Pareto front. Table 9 presents the comparative results across different problem settings in terms of both average performance and compromise solutions. The results reveal that as the problem scale increases, such as through a greater number of initial jobs and new orders, the performance differences among the model architectures become more pronounced. The proposed full model consistently exhibits the most stable scheduling performance across various problem scales, maintaining the lowest makespan and cost even in large-scale scenarios, demonstrating strong generalisation and robustness. In contrast, Experiment I, which excludes the SHGAT module entirely and relies solely on raw node attributes as input, exhibits substantial performance degradation with increasing problem complexity. This result underscores the critical importance of the graph embedding mechanism for capturing complex dependencies and facilitating effective policy learning. While Experiments II and III retain partial graph structural modelling, by including either relation-aware or node-aware components, they both suffer from notable information loss due to the absence of local attention mechanisms. This limitation becomes more pronounced in larger problem instances, where the diversity and density of scheduling information require finer semantic discrimination. These findings confirm the effectiveness and adaptability of the proposed SHGAT module in handling scheduling problems of varying scales in dynamic manufacturing environments.

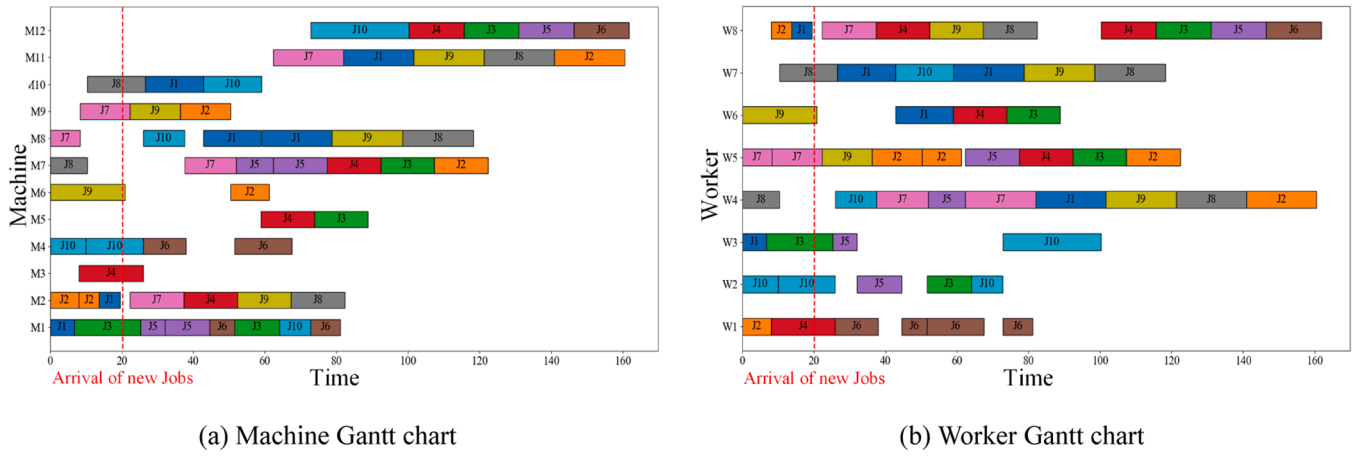


Fig. 14. Gantt chart of the initial scheduling plan.

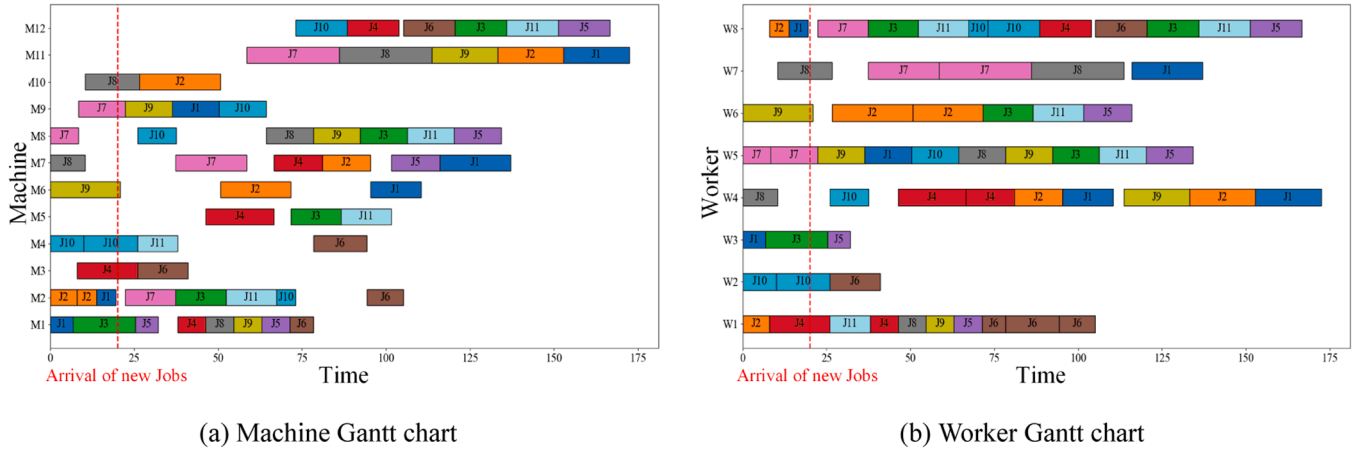


Fig. 15. Gantt chart of the rescheduling plan after the arrival of a new order.

5.3.2. Comparisons with other methods

To further validate the effectiveness and superiority of the proposed MA-PPO algorithm, a series of comparative experiments were conducted under different conditions, such as different initial job counts and dynamic job arrival rates. Six composite rules, PPO and DDQN were selected as baseline algorithms for comparison. Among them, PPO shares the same semantic feature extraction mechanism as MA-PPO. For DDQN, the global vector sv_t is used as the state input, and the decision-making follows the rule sets, as detailed in Table 10. The composite rules are constructed by pairing effective process sequencing strategies, including Shortest Processing Time (SPT), Most Work Remaining (MWKR), and Most Operations Remaining (MOR), with well-established worker assignment rules, including Lowest Worker Cost (LWC) [29] and Highest Proficiency Level (HPL) [30]. Machine assignments across all composite rule configurations are uniformly handled using the SPT rule to maintain consistency.

Table 11 presents the comparative performance of the three algorithms, MA-PPO, PPO, and DDQN, on the Pareto-optimal solution sets under different configurations of new and initial workpieces. The best results in each category are highlighted in bold. Across most experimental settings, MA-PPO consistently outperforms both PPO and DDQN, achieving lower GD values, better solution diversity, and lower IGD values. These results demonstrate the effectiveness of the proposed multi-action structure embedded in the MA-PPO framework. The separate action spaces allow the agent to dynamically select the optimal process, machine, and worker based on the current environment, and to achieve a well-balanced trade-off between makespan and cost through weighted reward composition. Notably, as the problem complexity increases, the performance gap between MA-PPO and baselines becomes more pronounced, highlighting its adaptability and optimisation capability in complex manufacturing environments.

Table 12 presents the comparative experimental results between the proposed method and various composite rule-based scheduling approaches, with the best results across all algorithms are highlighted in bold. Across all 12 experimental configurations, the proposed method consistently achieves the best performance on all three evaluation metrics, GD, IGD, and Spread, demonstrating its overall superiority in scheduling effectiveness. For example, under the configuration with $n = 20$, $\beta_p = 20$ and $n_{new} = 10$, the proposed method achieves a GD of 0.0190, IGD of 0.0142, and Spread of 0.6246, outperforming all composite rule-based methods. In particular, with respect to solution diversity (Spread), it significantly surpasses rule combinations such as R2 (0.8954) and R5 (0.9290). As the problem scale increases, traditional rule-based methods generally exhibit higher GD and IGD values and poor distribution in Spread. In contrast, the proposed method consistently maintains optimal performance, reflecting its strong generalisation ability and robustness in large-scale, dynamic conditions. Overall, these results validate that the proposed algorithm exhibits superior convergence capability, solution distribution stability, and environmental adaptability in multi-objective scheduling problems.

Fig. 11 illustrates the Pareto fronts obtained by the three algorithms, under scenarios involving varying numbers of initial workpieces and the dynamic addition of 10 new jobs. Across all instances, the Pareto fronts exhibit relatively uniform distributions, indicating good solution

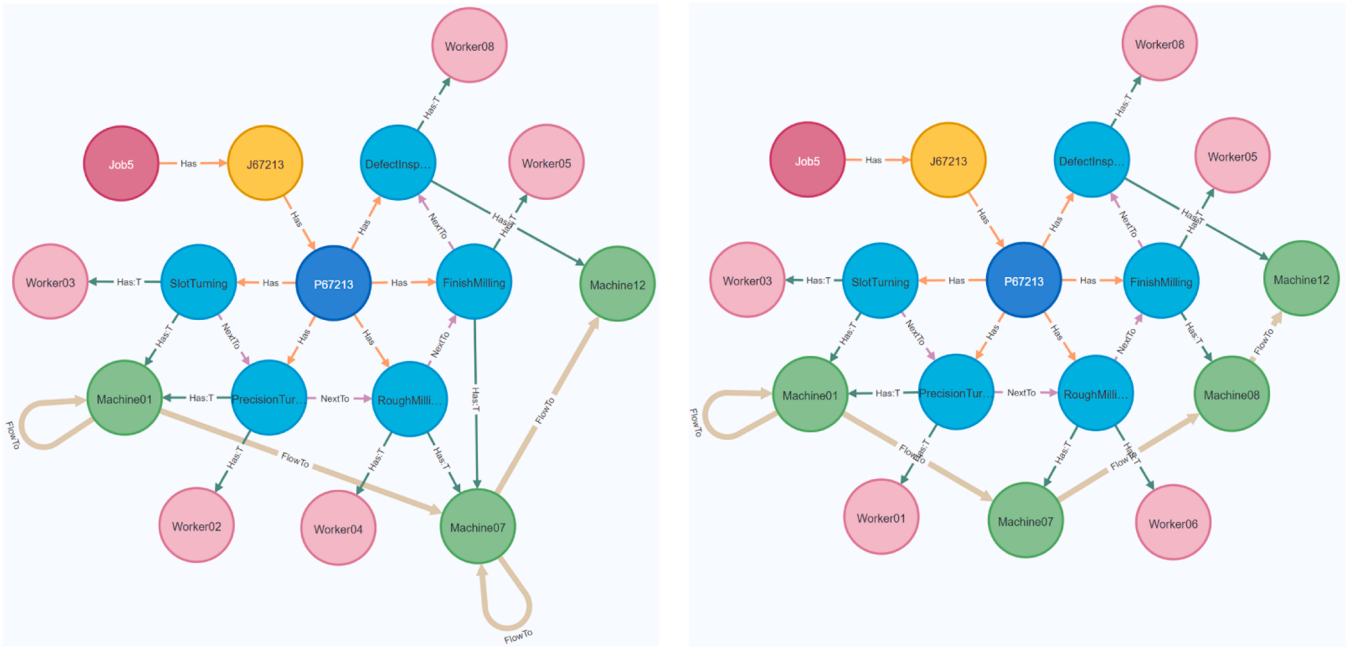


Fig. 16. Processing flow of J5 in the MPDKG.

diversity. However, PPO consistently outperforms DDQN, primarily because DDQN's action space heavily relies on the quality of its internal rule set, which limits the flexibility of the policy and constrains its global exploration ability. In contrast, PPO dynamically generates diverse PRDs at each decision step based on the probability distribution output by its policy network, resulting in a broader search space and higher adaptability. Overall, MA-PPO, with its multi-actor policy network structure, achieves a well-balanced trade-off between makespan and cost optimisation and demonstrates superior performance and robustness as task scale increases.

Fig. 12 (a) and (b) present the histograms of average makespan and average cost obtained by MA-PPO, PPO, and DDQN. The results clearly show that MA-PPO achieves superior performance on both objective functions as compared to PPO and DDQN across various problem sizes.

As shown in the Fig. 13, the comparison of computation time across different problem scales demonstrates that reinforcement learning-based methods can maintain second-level response speeds, effectively meeting the real-time requirements of practical manufacturing environments. The proposed MA-PPO method incorporates additional network modules, specifically, multiple Actor components, while maintaining overall computational efficiency. This results in a slightly increased computation time relative to PPO. However, this computational overhead is acceptable, especially considering the significant improvements in scheduling performance brought by these modules. Moreover, in the majority of the experimental configurations, the proposed method still exhibits lower computation time than DDQN, with this advantage becoming more evident as the problem scale increases. This indicates that the proposed approach achieves a favourable balance between scheduling performance and computational efficiency, making it feasible and practical for real-time deployment in dynamic manufacturing environments.

After validating the algorithms, the MA-PPO model, selected for its superior generalisation performance in the test cases, is applied to generate a scheduling solution set for the aerospace manufacturing workshop at a time t with a production scale of $10\times12\times8$. The workpieces involved in the case study belong to six different types: J1 and J2 are of type *J12769*; J3 and J4 are of type *J34892*; J5 is of type *J67213*; J6 is of type *J98245*; and J7, J8, and J9 are of type *J73901*, whereas J10 belongs to type *J68913*. Based on the obtained trade-off solution, the initial scheduling plan yields a makespan of 161.78 min and a total cost of RMB 3,887.77. The machine Gantt chart and worker Gantt chart are shown in Fig. 14 (a) and (b), respectively, where the horizontal axis represents processing time, and the vertical axis indicates the machine and worker indices.

At production time 20 (indicated by the red dashed line), a new high-priority workpiece J11 (of type *J34892*) arrives, superseding the priority of all currently scheduled tasks. The rescheduling solution is illustrated in the machine and worker Gantt charts shown in Fig. 15 (a) and (b), respectively. The updated schedule results in a makespan of 172.49 min and a total cost of RMB 4,063.63.

Dynamic task arrivals can lead to changes in the processing sequences and resource assignments of jobs. The MPDKG is automatically updated in real time to reflect the revised scheduling plan. Taking job J5 (of type *J67213*) as an illustrative example, its process flow is shown in Fig. 16. The initial process flow of J5 in the MPDKG after executing the original scheduling plan is shown in Fig. 16 (a). After applying the rescheduling strategy, the MPDKG is updated accordingly, and the revised process flow of J5 is illustrated in Fig. 16 (b). A comparative analysis indicates several changes: the operator for the PrecisionTurning-1 operation has been changed from Worker02 to Worker01, the RoughMilling operation has been adjusted from Worker04 to Worker06, and the machine assigned to FinishMilling has been updated from Machine07 to Machine08.

The MPDKG provides semantically enriched, real-time state information as input to the MA-PPO algorithm, enabling the generation of adaptive and context-aware scheduling solutions. Upon obtaining the scheduling results, the MPDKG is dynamically updated based on the latest execution status. This knowledge-driven closed-loop feedback mechanism between the perception and decision-making modules effectively enhances the responsiveness and adaptability of the scheduling system under frequent disturbances and changing shop-floor conditions.

6. Conclusions and future work

This study addresses the DRCFJSP by incorporating key human-centric factors, such as worker fatigue and skill variability, into the scheduling model. To tackle the complexities of real-time decision-making in dynamic manufacturing environments, a hybrid scheduling approach is proposed, integrating KG and DRL. The proposed method is structured into two main modules. First, a knowledge fusion module is developed using a multi-level semantic representation model to capture relationships among workers, machines, and operations. An MPDKG is designed, detailing its construction, semantic structure, and dynamic update mechanism. By embedding scheduling-related entities and semantic relations, the MPDKG provides a unified and dynamic representation of shop-floor information, enabling data-driven optimisation. Second, a DRL-based dynamic scheduling module is introduced to leverage the knowledge embedded in MPDKG. A SHGAT is constructed to perform semantic feature fusion and extract expressive embeddings for scheduling entities. These embeddings serve as input to an MA-PPO algorithm, which coordinates scheduling decisions across operations, machines, and workers in real time. Case studies demonstrate that the proposed approach exhibits strong environmental responsiveness, improves scheduling efficiency and workshop productivity, and reduces overall production costs, offering an innovative and efficient solution for dynamic scheduling in DRC flexible job shops. Although the experiments focus on an aerospace manufacturing workshop, the proposed framework is domain-agnostic and can be readily extended to other complex manufacturing sectors, such as automotive and electronics, where real-time, resource- constrained scheduling is critical.

For future research, we plan to incorporate additional cognitive factors, such as learning and forgetting effects, to further enhance the worker modelling component. The method will also be extended to more complex shop-floor environments by integrating additional manufacturing constraints. Furthermore, we will explore the application of the MPDKG in fault prediction and quality causality analysis, and develop lightweight network architectures to reduce computational complexity, thereby unlocking the full optimisation potential of DRL in intelligent workshop scheduling.

CRediT authorship contribution statement

Yiwen Hu: Writing – original draft, Software, Methodology. **Jie Chen:** Writing – review & editing, Supervision. **Zequn Zhang:** Supervision, Conceptualization. **Dunbing Tang:** Project administration, Conceptualization. **Qixiang Cai:** Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work was supported by the National Natural Science Foundation of China (92267109, 52305539), the Natural Science Foundation of Jiangsu Province (BK20230880), the Fundamental Research Funds for the Central Universities (NS2024033), the National Social Science Fund of China (21BTQ107), and the Aeronautical Science Foundation of China (2023Z074052007).

Appendix

Table 1
Knowledge nodes and semantic feature representation of semantic relationships.

Category	Representation	Explanation of semantic features
Operation knowledge node	$g_i^o \in \mathbb{R}^{d_o}$	Selection of machine count. The number of workers selected. State of adjustment:adjusted is 1, unadjusted is 0. Processing time: adjusted is p_{ijkr}, unadjusted is $\overline{p}_{ij}$. Estimated or actual start time. The number of unadjusted operations of the job. The estimated or actual completion time of the job.
Machine knowledge node	$g_i^m \in \mathbb{R}^{d_M}$	Unit time processing cost. Machine coefficient. The number of operations that can be processed. Available time. Utilization rate: Processing time to production time ratio.
Worker knowledge node	$g_i^w \in \mathbb{R}^{d_W}$	Worker coefficient. Fatigue level. The number of operations that can be processed. Available time. Utilization rate: Processing time to production time ratio.
Machine-operation relation	$u_{ij}^{r^{mo}} \in \mathbb{R}^{d_{r^{mo}}}$	Preset processing time of the process on the machine.
Worker-operation relation	$u_{ij}^{r^{wo}} \in \mathbb{R}^{d_{r^{wo}}}$	Worker proficiency in processing the operation. Unit time processing cost.

Data availability

Data will be made available on request.

Reference

[1] D. Romero, J. Stahre, Towards the resilient operator 5.0: the future of work in smart resilient manufacturing systems, Procedia CIRP 104 (2021) 1089–1094, https://doi.org/10.1016/j.procir.2021.11.183.

[2] Y. Lu, H. Zheng, S. Chand, W. Xia, Z. Liu, X. Xu, L. Wang, Z. Qin, J. Bao, Outlook on human-centric manufacturing towards industry 5.0, J. Manuf. Syst. 62 (2022) 612–627, https://doi.org/10.1016/j.jmsy.2022.02.001.
[3] J. Xu, X. Xu, S. Xie, Recent developments in Dual Resource Constrained (DRC) system research, Eur. J. Oper. Res. 215 (2011) 309–318, https://doi.org/10.1016/j.ejor.2011.03.004.
[4] K. Han, W. Gong, Memetic algorithm based on non-dominated levels for flexible job shop scheduling problem with learn-forgetting effect and worker cooperation, Comput. Ind. Eng. 200 (2025) 110845, https://doi.org/10.1016/j.cie.2024.110845.
[5] F. Yu, C. Lu, J. Zhou, L. Yin, Mathematical model and knowledge-based iterated greedy algorithm for distributed assembly hybrid flow shop scheduling problem with dual-resource constraints, Expert Syst. Appl. 239 (2024) 122434, https://doi.org/10.1016/j.eswa.2023.122434.
[6] X. Guo, L. Zhou, Z. Zhang, L. Qi, J. Wang, S. Qin, J. Cao, Multi-objective optimization of multi-product parallel disassembly line balancing problem considering multi-skilled workers using a discrete chemical reaction optimization algorithm, Comput. Mater. Contin. 80 (2024) 4475–4496, https://doi.org/10.32604/cmc.2024.048123.
[7] P. Zheng, L. Xia, C. Li, X. Li, B. Liu, Towards self-X cognitive manufacturing network: an industrial knowledge graph-based multi-agent reinforcement learning approach, J. Manuf. Syst. 61 (2021) 16–26, https://doi.org/10.1016/j.jmsy.2021.08.002.
[8] B. Liu, C. Chen, Z. Wang, A multi-hierarchical aggregation-based graph convolutional network for industrial knowledge graph embedding towards cognitive intelligent manufacturing, J. Manuf. Syst. 76 (2024) 320–332, https://doi.org/10.1016/j.jmsy.2024.08.012.
[9] B. Zhou, J. Bao, J. Li, Y. Lu, T. Liu, Q. Zhang, A novel knowledge graph-based optimization approach for resource allocation in discrete manufacturing workshops, Robot. Comput. Integr. Manuf. 71 (2021) 102160, https://doi.org/10.1016/j.rcim.2021.102160.
[10] Y. Liu, Research on Dual-System Knowledge Modeling and Scheduling Methods for Flexible Job Shop Scheduling, Donghua University, 2023, https://doi.org/10.27012/d.cnki.gdhuu.2022.001454. Master's Thesis.
[11] R. Hedman, R. Sundkvist, P. Almström, A. Kinnander, Object-oriented modeling of manufacturing resources using work study inputs, Procedia CIRP 7 (2013) 443–448, https://doi.org/10.1016/j.procir.2013.06.013.
[12] H. Li, D. Yang, H. Cao, W. Ge, E. Chen, X. Wen, C. Li, Data-driven hybrid petri-net based energy consumption behaviour modelling for digital twin of energy-efficient manufacturing system, Energy 239 (2022) 122178, https://doi.org/10.1016/j.energy.2021.122178.
[13] C. Yang, Y. Zheng, X. Tu, R. Ala-Laurinaho, J. Autiosalo, O. Seppänen, K. Tammi, Ontology-based knowledge representation of industrial production workflow, Adv. Eng. Inf. 58 (2023) 102185, https://doi.org/10.1016/j.aei.2023.102185.
[14] Z. Jiang, B. Xie, S. Zhu, H. Zhang, W. Yan, M. Dai, Method of building remanufacturing disassembly process knowledge graph driven by dynamic data flow, Comput. Integr. Manuf. Syst. 30 (2024) 879, https://doi.org/10.13196/j.cims.2023.0377.
[15] E.M. Sanfilippo, W. Terkaj, S. Borgo, Ontological modeling of manufacturing resources, AO 16 (2021) 87–109, https://doi.org/10.3233/AO-210242.
[16] Q. Bao, G. Zhao, Y. Yu, S. Dai, W. Wang, The ontology-based modeling and evolution of digital twin for assembly workshop, Int. J. Adv. Manuf. Technol. 117 (2021) 395–411, https://doi.org/10.1007/s00170-021-07773-1.
[17] B. Xiao, B. Xu, S. Peng, Y. Wei, G. Ding, M. Wang, Z. Zhao, Knowledge modeling of complex thin-walled parts machining process based on knowledge graph, Aeronaut. Manuf. Technol. 67 (2024) 76–86, https://doi.org/10.16080/j.issn1671-833x.2024.11.076.
[18] Y. Duan, L. Hou, S. Leng, A novel cutting tool selection approach based on a metal cutting process knowledge graph, Int. J. Adv. Manuf. Technol. 112 (2021) 3201–3214, https://doi.org/10.1007/s00170-021-06606-5.
[19] X. Zhang, X. Liu, X. Li, D. Pan, MMKG: an approach to generate metallic materials knowledge graph based on DBpedia and Wikipedia, Comput. Phys. Commun. 211 (2017) 98–112, https://doi.org/10.1016/j.cpc.2016.07.005.
[20] P. Kumar, S. Gupta, Y.R. Gunda, Estimation of human error rate in underground coal mines through retrospective analysis of mining accident reports and some error reduction strategies, Saf. Sci. 123 (2020) 104555, https://doi.org/10.1016/j.ssci.2019.104555.
[21] H. Song, J. Li, Z. Du, X. Yu, Y. Xu, Z. Zheng, J. Li, A Q-learning driven multi-objective evolutionary algorithm for worker fatigue dual-resource-constrained distributed hybrid flow shop, Comput. Oper. Res. 175 (2025) 106919, https://doi.org/10.1016/j.cor.2024.106919.
[22] Z. Ren, H. Qu, M. Safran, J. Wan, Dual-resource optimization configuration and collaborative scheduling for flexible job shop under worker flexibility constraints, IEEe Access. (2025), https://doi.org/10.1109/ACCESS.2025.3562754.
[23] N. Berti, S. Finco, O. Battaïa, X. Delorme, Ageing workforce effects in dual-resource constrained job-shop scheduling, Int. J. Prod. Econ. 237 (2021) 108151, https://doi.org/10.1016/j.ijpe.2021.108151.
[24] Q. Luo, Q. Deng, G. Xie, G. Gong, A Pareto-based two-stage evolutionary algorithm for flexible job shop scheduling problem with worker cooperation flexibility, Robot. Comput. Integr. Manuf. 82 (2023) 102534, https://doi.org/10.1016/j.rcim.2023.102534.
[25] L. Meng, C. Zhang, Y. Ren, B. Zhang, C. Lv, Mixed-integer linear programming and constraint programming formulations for solving distributed flexible job shop scheduling problem, Comput. Ind. Eng. 142 (2020) 106347, https://doi.org/10.1016/j.cie.2020.106347.
[26] J.R. Zamiska, M.Y. Jaber, H.V. Kher, Worker deployment in dual resource constrained systems with a task-type factor, Eur. J. Oper. Res. 177 (2007) 1507–1519, https://doi.org/10.1016/j.ejor.2005.04.018.
[27] J.a.C. Bokhorst, J. Slomp, G. Gaalman, On the who-rule in Dual Resource Constrained (DRC) manufacturing systems, Int. J. Prod. Res. (2004). https://www.semanticscholar.org/paper/On-the-who-rule-in-Dual-Resource-Constrained-(DRC)-Bokhorst-Slomp/6f1c31e761634b98160679bfdb43ed54962ca114 (accessed August 21, 2024).
[28] B.A. Norman, W. Tharmmaphornphilas, K.L. Needy, B. Bidanda, R.C. Warner, Worker assignment in cellular manufacturing considering technical and human skills, Int. J. Prod. Res. (2002), https://doi.org/10.1080/00207540110118082.
[29] Z. Zhang, M. Chica, Q. Tang, Z. Li, L. Zhang, A multi-objective co-evolutionary algorithm for energy and cost-oriented mixed-model assembly line balancing with multi-skilled workers, Expert. Syst. Appl. 236 (2024) 121221, https://doi.org/10.1016/j.eswa.2023.121221.
[30] Y. Liu, J. Fan, L. Zhao, W. Shen, C. Zhang, Integration of deep reinforcement learning and multi-agent system for dynamic scheduling of re-entrant hybrid flow shop considering worker fatigue and skill levels, Robot. Comput. Integr. Manuf. 84 (2023) 102605, https://doi.org/10.1016/j.rcim.2023.102605.
[31] F. Yu, C. Lu, L. Yin, J. Zhou, Modeling and optimization algorithm for energy-efficient distributed assembly hybrid flowshop scheduling problem considering worker resources, J. Ind. Inf. Integr. (2024) 100620, https://doi.org/10.1016/j.jii.2024.100620.
[32] S. Zhao, H. Zhou, Y. Zhao, D. Wang, DQL-assisted competitive evolutionary algorithm for energy-aware robust flexible job shop scheduling under unexpected disruptions, Swarm. Evol. Comput. 91 (2024), https://doi.org/10.1016/j.swevo.2024.101750.
[33] S.R. Kamali, T. Banirostam, H. Motameni, M. Teshnehlab, An immune-based multi-agent system for flexible job shop scheduling problem in dynamic and multi-objective environments, Eng. Appl. Artif. Intell. 123 (2023) 106317, https://doi.org/10.1016/j.engappai.2023.106317.
[34] Y. Zeiträg, J.R. Figueira, N. Horta, R. Neves, Surrogate-assisted automatic evolving of dispatching rules for multi-objective dynamic job shop scheduling using genetic programming, Expert. Syst. Appl. 209 (2022) 118194, https://doi.org/10.1016/j.eswa.2022.118194.
[35] C. Zhang, W. Song, Z. Cao, J. Zhang, P.S. Tan, X. Chi, Learning to dispatch for job shop scheduling via deep reinforcement learning. Advances in Neural Information Processing Systems, Curran Associates, Inc., 2020, pp. 1621–1632, in: https://proceedings.neurips.cc/paper_files/paper/2020/hash/11958dfee29b6709f48a9ba0387a2431-Abstract.html (accessed March 24, 2025).
[36] S. Luo, Dynamic scheduling for flexible job shop with new job insertions by deep reinforcement learning, Appl. Soft. Comput. 91 (2020) 106208, https://doi.org/10.1016/j.asoc.2020.106208.
[37] J. Chang, D. Yu, Y. Hu, W. He, H. Yu, Deep reinforcement learning for dynamic flexible job shop scheduling with random job arrival, Processes 10 (2022) 760, https://doi.org/10.3390/pr10040760.
[38] Z. Liu, H. Mao, G. Sa, H. Liu, J. Tan, Dynamic job-shop scheduling using graph reinforcement learning with auxiliary strategy, J. Manuf. Syst. 73 (2024) 1–18, https://doi.org/10.1016/j.jmsy.2024.01.002.
[39] K. Lei, P. Guo, Y. Wang, J. Zhang, X. Meng, L. Qian, Large-scale dynamic scheduling for flexible job-shop with random arrivals of new jobs by hierarchical reinforcement learning, IEEe Trans. Industr. Inform. 20 (2024) 1007–1018, https://doi.org/10.1109/TII.2023.3272661.
[40] M. Zhang, L. Wang, F. Qiu, X. Liu, Dynamic scheduling for flexible job shop with insufficient transportation resources via graph neural network and deep reinforcement learning, Comput. Ind. Eng. 186 (2023) 109718, https://doi.org/10.1016/j.cie.2023.109718.
[41] A. Ferjani, A. Ammar, H. Pierreval, S. Elkosantini, A simulation-optimization based heuristic for the online assignment of multi-skilled workers subjected to fatigue in manufacturing systems, Comput. Ind. Eng. 112 (2017) 663–674, https://doi.org/10.1016/j.cie.2017.02.008.
[42] J.A. Kent-Braun, A.V. Ng, J.W. Doyle, T.F. Towse, Human skeletal muscle responses vary with age and gender during fatigue due to incremental isometric exercise, J. Appl. Physiol. 93 (2002) 1813–1823, https://doi.org/10.1152/japplphysiol.00091.2002 (1985).
[43] M.Y. Jaber, Z.S. Givi, W.P. Neumann, Incorporating human fatigue and recovery into the learning–forgetting process, Appl. Math. Model. 37 (2013) 7287–7299, https://doi.org/10.1016/j.apm.2013.02.028.
[44] S. Konz, Work/rest: part II – The scientific basis (knowledge base) for the guide1. Elsevier Ergonomics Book Series, Elsevier, 2000, pp. 401–427, https://doi.org/10.1016/S1572-347X(00)80033-6.
[45] R. Brauchler, K. Landau, Task analysis: part II—The scientific basis (knowledge base) for the guide 1, Int. J. Ind. Ergon. 22 (1998), https://doi.org/10.1016/S1572-347X(00)80003-8.
[46] Y. Soo, M. Nishino, M. Sugi, H. Yokoi, T. Arai, R. Kato, T. Nakamura, J. Ota, Evaluation of frequency band technique in estimating muscle fatigue during dynamic contraction task, in: Proceedings of the IEEE International Conference on Robotics and Automation, 2009, pp. 933–938, https://doi.org/10.1109/ROBOT.2009.5152845.
[47] Q. Wang, C. Li, Y. Zhang, Y. Liu, H. Xiong, T. Shen, Knowledge graph construction: state-of-the art techniques and future opportunities, in: Proceedings of the 12th International Conference on Information Systems and Computing Technology (ISCTech), 2024, pp. 1–7, https://doi.org/10.1109/ISCTech63666.2024.10845648.
[48] M. Ruta, F. Scioscia, F. Gramegna, S. Ieva, E. Di Sciascio, R. Perez De Vera, A knowledge fusion approach for context awareness in vehicular networks, IEEe

Internet Things J. 5 (2018) 2407–2419, https://doi.org/10.1109/JIOT.2018.2815009.
[49] A. Bordes, N. Usunier, A. Garcia-Duran, J. Weston, O. Yakhnenko, Translating embeddings for modeling multi-relational data. Advances in Neural Information Processing Systems, Curran Associates, Inc., 2013. https://papers.nips.cc/paper_files/paper/2013/hash/1cecc7a77928ca8133fa24680a88d2f9-Abstract.html (accessed December 9, 2024).
[50] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, (2017). 10.48550/arXiv.1707.06347.
[51] H.V. Hasselt, A. Guez, D. Silver, Deep reinforcement learning with double Q-learning, in: Proceedings of the AAAI Conference on Artificial Intelligence, 2015, https://doi.org/10.1609/aaai.v30i1.10295.
[52] W. Song, X. Chen, Q. Li, Z. Cao, Flexible job-shop scheduling via graph neural network and Deep reinforcement learning, IEEe Trans. Industr. Inform. 19 (2) (2023) 1600–1610, https://doi.org/10.1109/TII.2022.3189725. IEEE.
[53] Z. Yue, A method for group decision-making based on determining weights of decision makers using TOPSIS, Appl. Math. Model. 35 (2011) 1926–1936, https://doi.org/10.1016/j.apm.2010.11.001.
[54] E. Zitzler, K. Deb, L. Thiele, Comparison of multiobjective evolutionary algorithms: empirical results, Evol. Comput. 8 (2000) 173–195, https://doi.org/10.1162/106365600568202.
[55] K. Deb, A. Pratap, S. Agarwal, T. Meyarivan, A fast and elitist multiobjective genetic algorithm: NSGA-II, IEEE Trans. Evol. Comput. 6 (2002) 182–197, https://doi.org/10.1109/4235.996017.
[56] E. Zitzler, L. Thiele, Multiobjective evolutionary algorithms: a comparative case study and the strength Pareto approach, IEEE Trans. Evol. Comput. 3 (1999) 257–271, https://doi.org/10.1109/4235.797969.